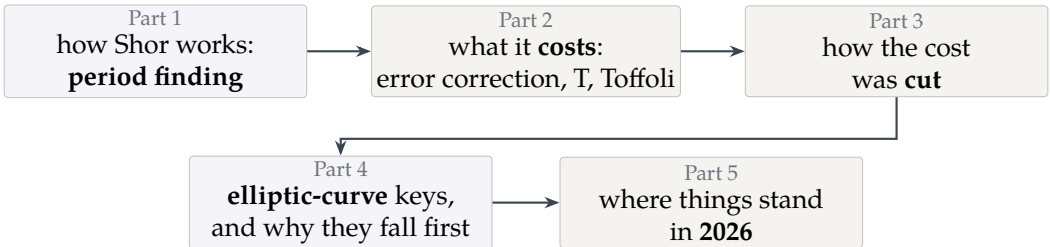
Shor’s Algorithm as Quantum Circuits

How factoring and the elliptic-curve discrete logarithm are built, run and costed — from the textbook circuit to the optimised ones, in Qiskit

---

A one-hour route through the full deck. Slide numbers are those of the full deck.

The road map for this hour



Each part answers one question, and each step uses only what came before it. Two background slides fill in what we need along the way: qubits and phases (now), error correction (Part 2).

> *Almost all of Shor's cost is reversible arithmetic: that is what we build, run and count.*

Part 1 — How Shor works

Background: qubits, gates and phases

States

A qubit is $\alpha|0\rangle + \beta|1\rangle$; measuring gives 0 with probability $|\alpha|^2$, 1 with $|\beta|^2$.

n qubits hold 2^n amplitudes, one per integer: $\sum_x \alpha_x |x\rangle$. Registers combine by $\otimes$: $|a\rangle \otimes |b\rangle = |a\rangle|b\rangle$.

Phases. A global phase $e^{i\theta}|\psi\rangle$ changes nothing. A **relative** phase, $\alpha|0\rangle + e^{i\theta}\beta|1\rangle$, is real: interference can read it.

Gates

H: $|0\rangle \mapsto \frac{|0\rangle+|1\rangle}{\sqrt{2}}$, $|1\rangle \mapsto \frac{|0\rangle-|1\rangle}{\sqrt{2}}$. X: NOT.

$P(\theta)$: $|1\rangle \mapsto e^{i\theta}|1\rangle$.

CNOT: $|a, b\rangle \mapsto |a, b \oplus a\rangle$. Controlled-$U$: apply U only where the control is 1.

QFT: $|x\rangle \mapsto \frac{1}{\sqrt{Q}} \sum_y e^{2\pi i\, xy/Q}|y\rangle$ writes x as a **phase ramp**; $\mathrm{QFT}^\dagger$ reads a ramp back as an integer.

Gates act on every branch of a superposition at once; the answer comes from interference, phases arranged so wrong answers cancel.

Act I — factoring reduces to finding a period

Difficulty 1. To factor N, pick a with $\gcd(a, N) = 1$ and find its **order** r: the smallest $r > 0$ with $a^r \equiv 1 \pmod{N}$.

x	0	1	2	3	4	5	$\cdots$
$7^x \bmod 15$	1	7	4	13	1	7	$\cdots$

$\Rightarrow\ r = 4$

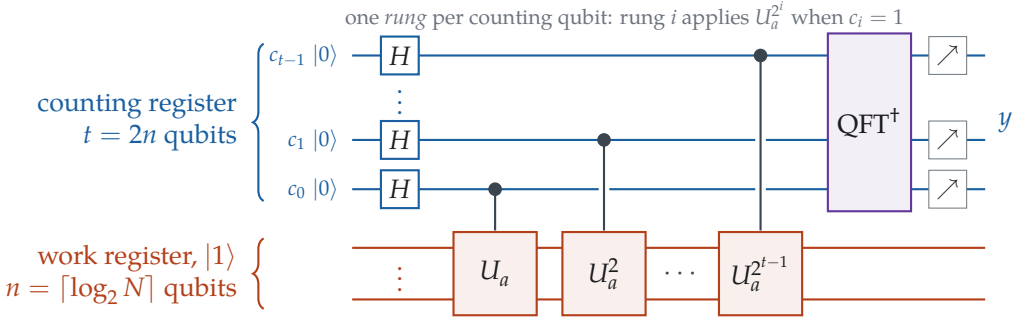
Why the period factors N: if r is even, N divides

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1),$$

and unless $a^{r/2} \equiv -1$ each factor shares a divisor with N: $\gcd(48, 15) = 3$, $\gcd(50, 15) = 5$.

> *The period is a property of the whole sequence, not of any one value. Only the period needs a quantum computer; the rest is Euclid.*

Phase estimation: the circuit



$$U_{\text{Shor}} = \left(\text{QFT}^\dagger \otimes I\right) \text{C-}U_a^{2^{t-1}} \cdots \text{C-}U_a^2 \ \text{C-}U_a \ \left(H^{\otimes t} \otimes I\right) \quad \text{applied to} \quad |0\rangle^{\otimes t}\,|1\rangle$$

where $\text{C-}U = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$, and $U_a^{2^i} = U_{a^{2^i} \bmod N}$ is computed classically.

Phase kickback: how the control learns the phase

If $|\psi\rangle$ is an eigenvector, $U|\psi\rangle = e^{2\pi i\varphi}|\psi\rangle$, a controlled-$U$ does this:

$$\text{C-}U\,(\alpha|0\rangle + \beta|1\rangle)|\psi\rangle \underset{\text{linearity}}{=} \alpha|0\rangle|\psi\rangle + \beta|1\rangle\, e^{2\pi i\varphi}|\psi\rangle \underset{\text{bilinearity}}{=} \big(\alpha|0\rangle + e^{2\pi i\varphi}\beta|1\rangle\big)|\psi\rangle.$$

Why

Linearity: only the $|1\rangle$ branch gets the phase. Bilinearity: a number on a product belongs to neither factor, $(\lambda a)\otimes b = a\otimes(\lambda b)$. Global on the target, so invisible; **relative** on the control, so readable.

Check on two qubits

CNOT on $|+\rangle|-\rangle$ gives $|-\rangle|-\rangle$: the "target" is unchanged and the "control" flipped, because $X|-\rangle = -|-\rangle$. The phase belongs to the pair.

In Shor, rung j kicks $e^{2\pi i\, 2^j s/r}$ onto counting qubit j: together, the phase ramp $\sum_x e^{2\pi i\, xs/r}|x\rangle$.

Why the measurement reveals s/r

1. **Eigenvectors.** $U_a|u_s\rangle = e^{2\pi i\,\varphi}|u_s\rangle$, $\varphi = s/r$; the input is $|1\rangle = \frac{1}{\sqrt{r}}\sum_s |u_s\rangle$.

2. **Kickback.** Rung j leaves $|u_s\rangle$ alone and turns its control by $2^j\varphi$. The counting register ($Q = 2^t$) becomes a product:

$$\bigotimes_j \frac{|0\rangle + e^{2\pi i\,2^j\varphi}|1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{Q}}\sum_{x=0}^{Q-1} e^{2\pi i\,\varphi\,x}\,|x\rangle =: |\Phi_\varphi\rangle$$

   since branch x collects the turns of its 1-bits, $\prod_j e^{2\pi i\,2^j\varphi\,x_j} = e^{2\pi i\,\varphi\,x}$: a *phase ramp* of slope φ.

3. **Readout.** With $\langle y|\,\mathrm{QFT}^\dagger|x\rangle = e^{-2\pi i\,xy/Q}/\sqrt{Q}$:

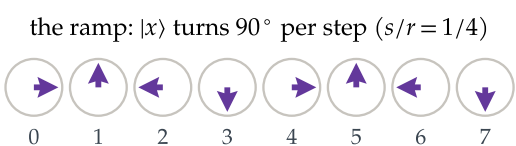
$$\langle y|\,\mathrm{QFT}^\dagger|\Phi_\varphi\rangle = \frac{1}{Q}\sum_x e^{-2\pi i\,xy/Q}\,e^{2\pi i\,\varphi x} = \frac{1}{Q}\sum_{x=0}^{Q-1} e^{2\pi i\,(\varphi - y/Q)\,x} = \begin{cases} \approx 1, & y \approx Q\varphi \\ \approx 0, & \text{otherwise} \end{cases}$$

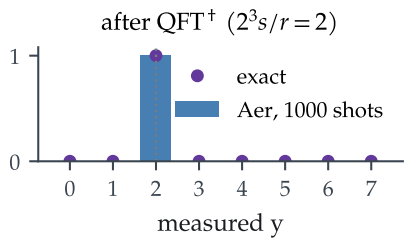
   Each $|u_s\rangle$ carries its own ramp; they do not interfere: $y \approx Q\,s/r$, s uniformly random.

$\mathrm{QFT}^\dagger$ in action: a ramp becomes an integer

$t = 3$, $s/r = 1/4$: the phase $e^{2\pi i x/4} = i^x$ turns a quarter turn per step:

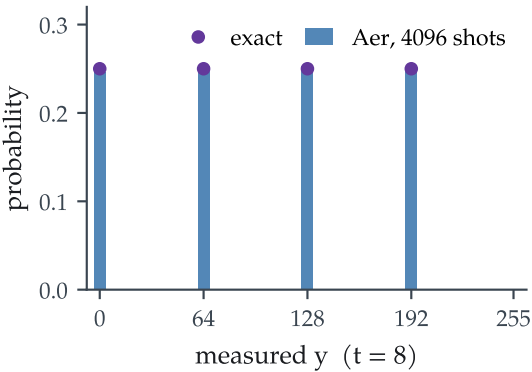
$$\frac{1}{\sqrt{8}} \sum_{x=0}^{7} e^{2\pi i x/4} |x\rangle = \frac{|0\rangle + i|1\rangle - |2\rangle - i|3\rangle + |4\rangle + \cdots}{\sqrt{8}} = \mathrm{QFT}\,|2\rangle \xrightarrow{\mathrm{QFT}^\dagger} |2\rangle$$





> *Every shot gives $y = 2$: $s/r = 2/8 = 1/4$, the ramp's slope read as an integer.*

$N = 15$: four sharp peaks



The exact distribution

The target takes r values, one per residue $x_0 = x \bmod r$:

$$P(y) = \frac{1}{4^t} \sum_{x_0 < r} \left| \sum_m e^{-2\pi i\, mry/2^t} \right|^2$$

Aer's exact probabilities equal it.

Samples vs exact: the total variation distance $\frac{1}{2}\sum_y |\hat{p}(y) - p(y)|$ is 0.006. A perfect sampler exceeds 0.032 only 0.01% of the time at 4096 shots — the *shot-noise bound*.

Part 2 — What it costs

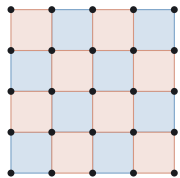
Background: error correction in one slide

Why. Physical qubits fail about once per 10^3 gates; Shor needs 10^8–10^{10} Toffolis.

Logical qubits. Store one protected qubit in many physical ones, e.g. $\alpha|000\rangle + \beta|111\rangle$. **Parity checks** ("are qubits 1 and 2 equal?") locate an error without reading α, β; a decoder fixes it.

Distance d, threshold. Below about 1% physical error, the logical error falls exponentially with d: $d \approx 27$ at $p = 10^{-3}$.

Cheap gates. *Transversal* gates act qubit by qubit, so one fault stays one error. On the surface code the Clifford gates (H, S, CNOT) are cheap; T is not.



Surface code, $d = 5$: dots are data qubits, squares are checks of Z (blue) or X (red) type on 4 neighbours. About $2d^2$ physical qubits per logical one: $\approx$1 500 at $d = 27$.

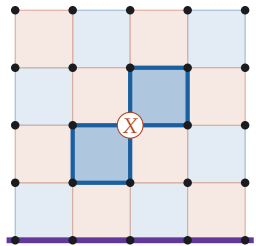
> *Error correction makes Clifford gates cheap and leaves one gate expensive: T.*

The surface code: parity checks, errors and the decoder

Measure parities, never the data. Every cycle (about 1 μs) each square measures the parity of its four corners: Z-squares catch bit flips, X-squares phase flips. No check reveals the stored state.

Find the errors. An error flips the squares next to it; a chain of errors flips only its two ends. A classical *decoder* pairs the flipped squares and infers the correction.

Where the qubit lives. The logical operator is a string across the whole patch (purple). Corrupting it takes d errors in a line: the distance.

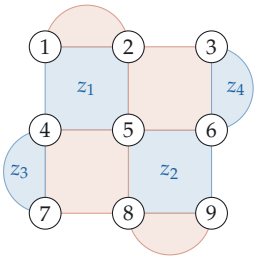


logical $\bar{X}$: a string of d

$d = 5$. One bit flip (centre) lights the two Z-squares beside it.

> *Below threshold* ($p_{\text{th}} \approx 1\%$), $P_L \approx (p/p_{\text{th}})^{(d+1)/2}$: *at* $p = 10^{-3}$, *each* $d \to d+2$ *divides it by about 10.*

The surface code as parity checks, at $d = 3$



9 data qubits; red X-checks catch Z errors, blue Z-checks catch X errors.

Z-checks as a parity-check matrix H_Z; an X error e gives $s = H_Z e$. Each check asks: *did an odd number of X errors hit my qubits?*

	1	2	3	4	5	6	7	8	9
z_1	1	1	0	1	1	0	0	0	0
z_2	0	0	0	0	1	1	0	1	1
z_3	0	0	0	1	0	0	1	0	0
z_4	0	0	1	0	0	1	0	0	0

Narrowing down. X_5 lights z_1, z_2: only qubit 5 is in both; X_7 lights z_3 alone.

Degenerate. X_1, X_2 give one syndrome, but X_1X_2 is a check: either fix works.

> $[[9, 1, 3]]$*: 8 checks, 1 logical qubit* ($\bar{X} = X_1X_4X_7$, $\bar{Z} = Z_1Z_2Z_3$)*, distance 3: every single error is fixed.*

How error correction runs: measure, decode, keep going

On the chip (quantum)

Every $\approx 1\,\mu s$ every check is measured, in parallel with the computation.

A check reads a parity, never the data: the computation **never stops**.

Logical gates are themselves check measurements (lattice surgery).

In the decoder (classical)

Watches for checks whose result **changes**: errors show in space, faulty measurements in time.

Pairs the changes into the most likely errors.

Records the correction in software (the *Pauli frame*); no gate is applied.

> *Never stop to correct. Wait only at T gates, whose correction needs the decoder's answer: the* $\approx 10\,\mu s$ *reaction time.*

Codes, their parameters, and the bounds

$[[n, k, d]]$: n physical qubits store k logical ones; d is the fewest single-qubit errors that change the data unseen. It corrects $\lfloor (d-1)/2 \rfloor$ errors.

code	$[[n, k, d]]$
five-qubit (smallest)	$[[5, 1, 3]]$
Steane	$[[7, 1, 3]]$
surface, rotated	$[[d^2, 1, d]]$
IBM gross (qLDPC)	$[[144, 12, 12]]$
good qLDPC (2021)	$[[n, \Theta(n), \Theta(n)]]$

How well can a code do?

Singleton: $n - k \geq 2(d-1)$.

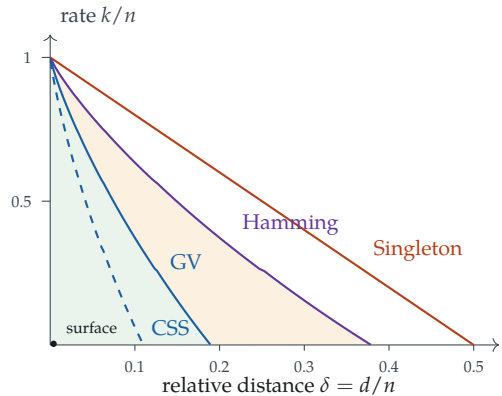
Hamming: $2^{n-k} \geq \sum_{j \leq t} \binom{n}{j} 3^j$; $[[5, 1, 3]]$ meets it ($16 = 1 + 15$).

Local checks on a flat chip: $k\,d^2 = O(n)$ (Bravyi–Poulin–Terhal). The surface code meets it.

Cheap gates: never universal (Eastin–Knill), so T needs magic states.

> *On a flat chip with local checks, about d^2 qubits per logical qubit is the best possible; doing better needs long-range links (qLDPC).*

Which codes can exist: the bounds



Large codes, rate k/n, relative distance $\delta = d/n$:

Singleton: $k/n \leq 1 - 2\delta$.

Hamming: $k/n \leq 1 - \frac{\delta}{2}\log_2 3 - H_2(\frac{\delta}{2})$; tighter than Singleton.

Gilbert–Varshamov: codes *exist* with $k/n \geq 1 - \delta \log_2 3 - H_2(\delta)$; CSS codes with $1 - 2H_2(\delta)$ (dashed).

Surface code: at the origin. Good qLDPC codes: constant k/n and d/n, with long-range links.

Green: codes exist. Peach: open. Above the Hamming curve: impossible for large codes.

From one patch to a computer

How big a patch

A Shor run has about 10^3 logical qubits for 10^9–10^{10} cycles, so it needs $P_L \approx 10^{-15}$ per qubit per cycle.

With a factor 10 per step: $d \approx 27$, so $2d^2 \approx 1\,500$ physical qubits per logical one, plus routing and factories.

Measured: on Google's Willow (2024) the logical error halved at each step $d = 3 \to 5 \to 7$.

How it computes

X, Z: free, tracked in software.

CNOT: **lattice surgery**. Merge two patches along an edge, measure their joint checks for about d cycles, split.

H, S: cheap, by similar tricks.

T: impossible this way (Eastin–Knill): a **magic state** from a factory, consumed by lattice surgery.

> *A machine is a floor of patches: memory, routing and magic-state factories. The Toffoli count sets the runtime.*

Why the T gate is the expensive one

Transversal gates

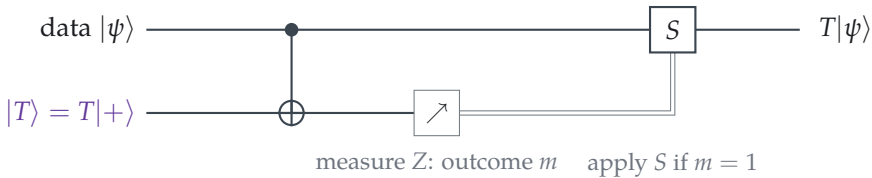
A *logical* qubit is spread over many physical qubits. The safest logical gate is **transversal**: the same gate on every physical qubit, so a fault cannot spread. On the surface code CNOT is transversal and H, S nearly so: Cliffords are cheap.

Eastin–Knill (2009)

No code has a transversal gate set that is *universal*. A code that ignores small local errors cannot let small local operations move its logical state. Some gate must be done another way: T.

> *Eastin–Knill makes the gate set finite; Solovay–Kitaev makes that affordable; magic states are the price of T.*

Magic states: T by teleportation



Why it works

After the CNOT, $m = 0$ leaves $T|\psi\rangle$ and $m = 1$ leaves $T^\dagger|\psi\rangle$; $S\,T^\dagger = T$. CNOT, measurement and S are all Clifford: the gate costs **one fresh** $|T\rangle$.

Factories

Regions of the chip that produce $|T\rangle$. *Distillation* (2005): 15 noisy copies $\to$ 1 with error $\approx 35\varepsilon^3$. *Cultivation* (2024): grow one copy inside a small code, checking as it grows; $\approx 10\times$ cheaper.

The Toffoli: a reversible AND

$$\text{Toffoli}\,|a,\,b,\,c\rangle = |a,\,b,\,c \oplus ab\rangle$$

Reversible: it permutes basis states

It swaps $|110\rangle \leftrightarrow |111\rangle$ and fixes the other six. A permutation matrix is unitary; its inverse is the reverse permutation, here the Toffoli itself:

$$c \oplus ab \oplus ab = c.$$

What it costs

In Clifford+T, a Toffoli is **7 T gates**. A classical function is therefore priced exactly by its Toffoli count.

> *Reversible arithmetic = Toffolis + CNOTs. Its T cost is exact: 7× the number of Toffolis.*

Irreversible functions as unitaries: the three tricks

1. Keep the input

Any f, even many-to-one:

$$U_f|x, y\rangle = |x,\ y \oplus f(x)\rangle$$

A permutation of basis states: unitary, its own inverse. Built from NOT, CNOT (XOR) and Toffoli (AND).

2. Clean the scratch

Gates leave garbage $g(x)$ (Bennett):

$$|x\rangle|0\rangle|0\rangle \to |x\rangle|f(x)\rangle|g(x)\rangle$$

copy $f(x)$ out, run backwards

Result: $|x\rangle$ and a clean copy of $f(x)$, for about $2\times$ the gates.

3. Invertible: in place

If f^{-1} is computable:

$$|x\rangle|0\rangle \to |x\rangle|f(x)\rangle \xrightarrow{\text{swap}} |f(x)\rangle|x\rangle \xrightarrow{f^{-1}} |f(x)\rangle|0\rangle$$

Shor: $\times a$, swap, $\times(-a^{-1})$ (slide 28).

Linearity does the rest: $\sum_x \alpha_x|x\rangle|0\rangle \mapsto \sum_x \alpha_x|x\rangle|f(x)\rangle$, every input at once.

> *Keep the input, XOR the output, clean the scratch; invertible f: swap, then erase x with f^{-1}.*

The price list

gate	T gates	note
H, S, CNOT (Clifford)	0	cheap on the code
$T = P(\pi/4)$	1	one magic state
Toffoli	7	(4 as a temporary AND, Act VII)
small rotation $P(\theta)$	$\approx 3\log_2(1/\varepsilon) \approx$ 60–100	budget ε/R for each of R

> ***Resolved.*** *One small rotation costs as much as about ten Toffolis. Circuits built from Toffolis have an exact, low T count; circuits built from rotations pay for synthesis on every one.*

Act III — arithmetic leaves scratch values behind

Difficulty 3. A quantum circuit cannot erase. Computing $f(x)$ reversibly leaves intermediate values — carries, flags, partial products — in **ancillas**, scratch qubits that start in $|0\rangle$. Does this *garbage* matter if we never measure it?

A reversible circuit for f:

$$|x\rangle\,|0\rangle\,|0\rangle \;\longmapsto\; |x\rangle\;|f(x)\rangle\;|g(x)\rangle,$$

where $g(x)$ is everything else it wrote down. In Shor, x is the counting register, in superposition:

$$|\Psi\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle\;|f(x)\rangle\;|g(x)\rangle, \qquad f(x) = a^x \bmod N, \quad Q = 2^t.$$

Garbage destroys the interference: the formula

Apply $\mathrm{QFT}^\dagger$ to the counting register, measure y. The other registers enter through their **overlaps**:

$$P(y) \;=\; \frac{1}{Q^2}\sum_{x,x'} e^{2\pi i\,(x'-x)\,y/Q}\;\underbrace{\langle f(x')|f(x)\rangle}_{\text{target}}\;\underbrace{\langle g(x')|g(x)\rangle}_{\text{garbage}}.$$

No garbage, or garbage set by $f(x)$

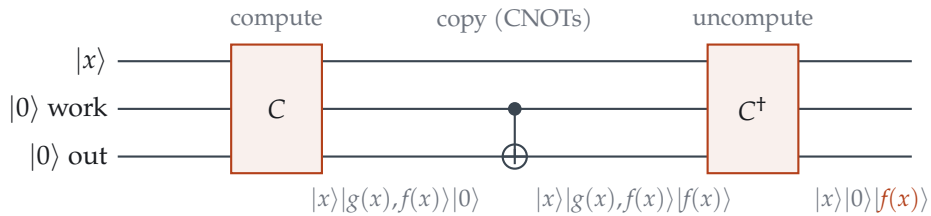
The overlaps are $[x \equiv x' \bmod r]$: each period's terms interfere, and $P(y)$ peaks at $y \approx sQ/r$.

Garbage that records x

$\langle g(x')|g(x)\rangle = \delta_{xx'}$: every cross term vanishes, and $P(y) = Q/Q^2 = 1/Q$ — flat.

> *Garbage is a which-path record. If it tells x apart beyond $f(x)$, it measures x — and the interference is gone.*

Bennett's fix: compute, copy, uncompute



The price

About twice the gates, plus a register for the copy: a factor 2 in every count that follows.

Where it returns

the modular adder's flag; multiply–swap–*un*-multiply; a point addition's second division; the temporary AND

> ***Resolved.*** *Every scratch register must be back in* $|0\rangle$ *before the QFT.*

Five levels, each a product of the one below

Difficulty 4. How does "multiply by a modulo N" become gates? Build it in levels, and check each level exactly before using it.

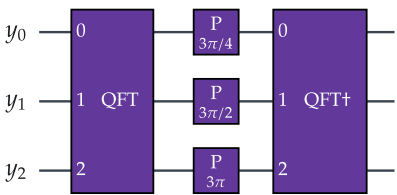
L5 U_{Shor}: t controlled rungs, then $\text{QFT}^\dagger$	`order_circuit`
L4 C-U_a: multiply, controlled swap, un-multiply	`c_ua`
L3 CMULT(a): n controlled modular additions of $a2^i$	`c_mult_acc`
L2 $\text{ADD}_N(X)$: seven blocks of plain additions	`c_add_mod`
L1 $\Phi\text{ADD}(X)$: QFT, then phases, then $\text{QFT}^\dagger$	`add_const`
L0 QFT: Hadamards and controlled phases	`qft`

Each level is verified exhaustively — every input, every ancilla back to $|0\rangle$ — before the next is built on it.

L1 — Draper's adder: addition is a phase

In the Fourier basis, qubit i of $\mathrm{QFT}|y\rangle$ carries $e^{2\pi i\, y 2^i/2^m}$. Adding X multiplies it by $e^{2\pi i\, X 2^i/2^m}$: one phase per qubit, **no carries**.

$$\Phi\mathrm{ADD}(X) = \mathrm{QFT}^\dagger \Big(\bigotimes_{i=0}^{m-1} P\big(2\pi X 2^i/2^m\big) \Big)\, \mathrm{QFT}$$

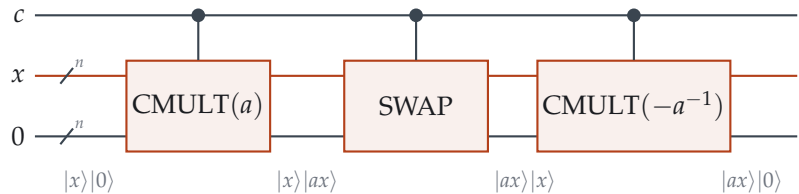


$X = 3$ on 3 qubits: checked equal to the cyclic shift $y \mapsto y + 3 \bmod 8$.

> *To control the adder, control only the phases: the QFT pair cancels when they do not fire.*

L3–4 — multiply, swap, un-multiply

Shift-and-add, $ax = \sum_i x_i\,(a2^i)$, gives an *out-of-place* product: CMULT(a) is n modular additions of $a2^i$, each controlled by c and x_i. Phase estimation needs U_a *in place*:



$$\text{C-}U_a = \text{CMULT}(-a^{-1}) \cdot \text{C-SWAP} \cdot \text{CMULT}(a)$$

> *One rung is* $2n$ *modular additions and* n *controlled swaps. The second multiplication only erases the input: uncomputation, made possible by* a^{-1}.

At RSA-2048 size

This circuit (verified formulas)

8 194 qubits, 6.9×10^{10} Toffolis
exact QFTs: $\approx 2 \times 10^{17}$ T
approximate QFTs: $\approx 1 \times 10^{16}$ T

still dominated by rotations

Reported by the state of the art

Gidney–Ekerå 2021: 6 189 qubits, 2.7×10^{9} Toffolis — ripple-carry, windowed.

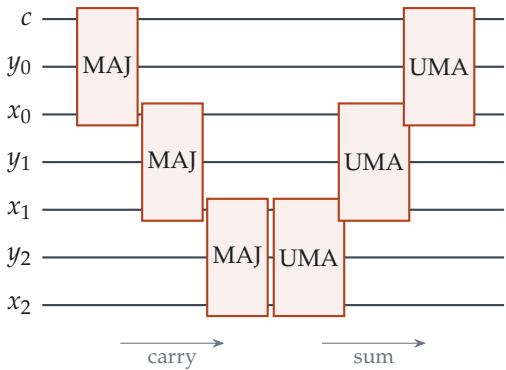
Gidney 2025: 1 399 qubits, 6.5×10^{9} Toffolis — approximate residue arithmetic.

Physically: $< 10^{6}$ qubits for 5 days, or $\approx$ 94 000 for a month.

> ***Resolved.*** *Polynomial but enormous, mostly rotations: Act VII removes them first.*

Part 3 — How the cost was cut

Toffoli arithmetic: the ripple-carry adder

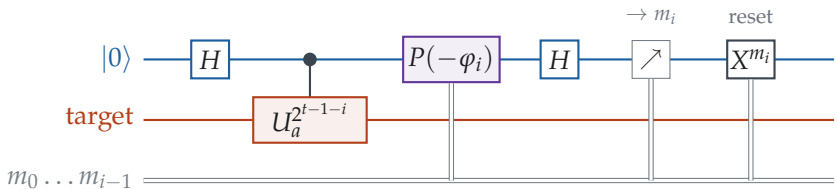


Cuccaro et al.: **MAJ** blocks carry up, **UMA** blocks write the sum on the way down. Each is one Toffoli and two CNOTs.

$2m$ Toffolis, one carry qubit, **no rotations**. The seven blocks of the modular adder stay the same; only the adder inside changes.

One counting qubit: the semiclassical QFT

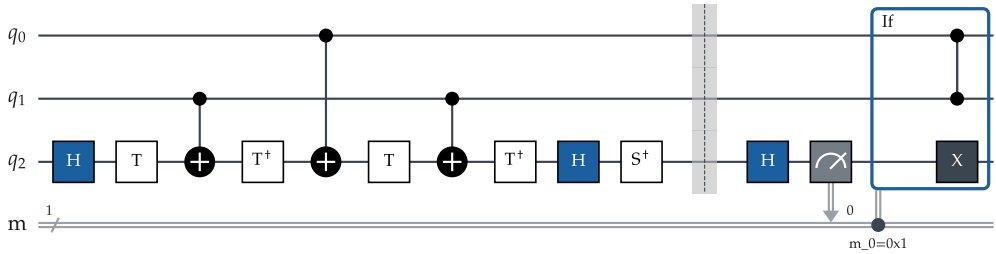
Every rotation of $\mathrm{QFT}^\dagger$ is controlled by a qubit *about to be measured*. Measure it first and control classically: the distribution is unchanged.



$$\varphi_i = \pi \sum_{j<i} m_j / 2^{i-j}, \qquad \text{repeat for } i = 0, \ldots, t-1 \text{ (heaviest rung first)}$$

> *One recycled qubit replaces all $2n$ counting qubits: $4n+2 \to 2n+3$. The controlled phases become classically controlled single-qubit phases.*

The temporary AND: uncomputation for free



Compute with 4 T; uncompute by an X-basis measurement and a classically controlled CZ.

Why it works

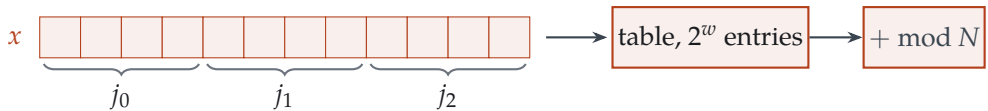
The target starts in $|0\rangle$: 4 T, not 7. An X-basis measurement reveals nothing about a, b; a Clifford fixes the phase.

What it saves

A compute/uncompute pair drops from 14 T to 4: a table lookup costs $3.5\times$ fewer T.

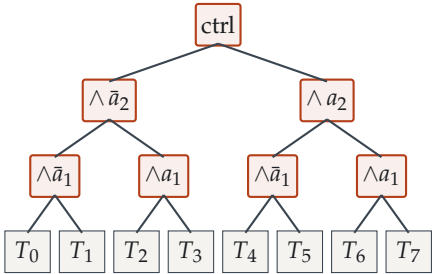
Windowing in three steps

1. **A controlled addition is a lookup.** Adding k when $q = 1$ is adding $T[q]$ from the table $T = [0, k]$.
2. **Address w bits at once.** Window j of x contributes $k\,j\,2^i \bmod N$: tabulate the 2^w values and add **once**, so n additions become $\lceil n/w \rceil$.
3. **Fold the control in.** The outer control is one more address bit whose $c = 0$ entries are 0: the modular adder needs **no** control.



> *Cost $O\big(\frac{n}{w}(n + 2^w)\big)$; $w \approx \log_2 n$ balances the terms: $O(n^2 / \log n)$, a gain that grows with n.*

Making the lookup cheap: unary iteration



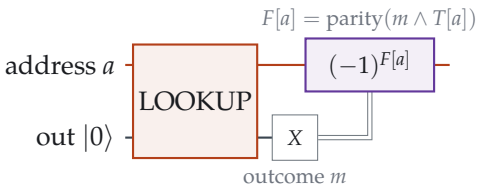
$m = 3$ address bits, $L = 8$ entries

The one-hot QROM holds all 2^m address states at once. Unary iteration walks the address **tree** instead, holding only the AND of the current prefix: a node is parent $\wedge\, a_i$ (one AND), its sibling the same with one CNOT, and each leaf loads $T[j]$ with CNOTs.

Measured

$2(L-1)$ Toffolis exactly $(2, 6, 14, 30$ for $m = 1..4)$ on m ancillas instead of 2^m — $L-1$ with temporary ANDs. At $N = 255$, $w = 4$: **59 qubits** $\rightarrow$ **31**, and the CNOT saving over shift-and-add grows from $1.67\times$ to $2.26\times$.

Uncomputing a lookup for free



Measuring the output in X gives a random m and leaves only the phase $(-1)^{m \cdot T[a]}$. A **phase** lookup costs $O(\sqrt{L})$: one-hot the low half of a, unary-iterate the high half applying CZ.

w	L	redo	measure	saving
2	4	6	3	2.0×
4	16	30	9	3.3×
8	256	510	45	11.3×
11	2 048	4 094	157	**26.1**×

Toffolis. Redo: $2(L-1)$; measure: $2^l - 1 + 2(2^h - 1)$, with $l = h = w/2$

At $w \approx 11$, as at cryptographic size, compute + uncompute costs 1.04 lookups, not 2.

Factoring: what each optimisation bought (1/2)

optimisation	what it removes	measured gain
Ripple-carry adders	every small rotation in the arithmetic	T -95% ($n = 4$) to -99% ($n = 12$)
Semiclassical QFT	the counting register: one qubit, reused	qubits -41% ($N = 21$), -50% at size
Control placement	controls on blocks that cancel anyway	CNOTs per adder -92%
Temporary AND	the T cost of uncomputing an AND	T per AND pair -71% ($14 \to 4$)
Windowing	most of the modular additions	additions -75%, depth -70%

Free before all of these: precomputed powers (t rungs, not $2^t - 1$) and controlled MAJ/UMA ($2n$ controlled gates, not $6n$). Windowing measured at $N = 255$, $w = 4$.

Factoring: what each optimisation bought (2/2)

optimisation	what it removes	measured gain
Unary iteration	the lookup's 2^w ancillas	lookup qubits -47% ($59 \to 31$)
Measured uncomputation	the second lookup	clean-up -96% ($4\,094 \to 157$)
Nested windows	exponent rungs and their controls	multiplications -50% ($8 \to 4$)
Coset representation	the modular reduction	Toffolis per addition -67 to -74%
Approximate QFT	the smallest rotations	rotations -34% ($n = 8$)

> *All together at $N = 15$: 978 000 $T \to$ **42 600** T (-96%), on 17 qubits instead of 18.*

Part 4 — Elliptic-curve keys

Act VIII — ECDSA keys are discrete logarithms

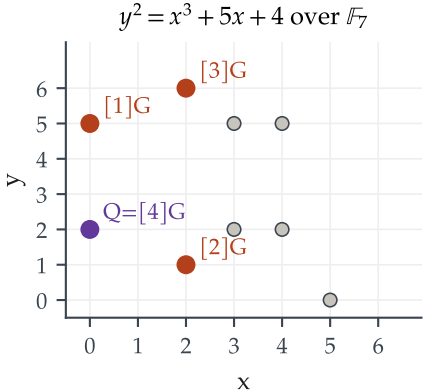
Difficulty 8. An ECDSA or ECDH key pair is a secret number d and a public point $Q = dG$ on a curve. Recovering d from Q is the elliptic-curve discrete logarithm. Does the same circuit solve it?

The points (x, y) with $y^2 = x^3 + ax + b \pmod{p}$, plus a point at infinity O, form a group. Adding two points:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \qquad x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1.$$

*Point addition needs a modular **division**. That is the whole cost difference from factoring.*

A toy curve



$y^2 = x^3 + 5x + 4$ over $\mathbb{F}_7$: nine points and O.

$kP = P + \cdots + P$ (k times). $G = (0, 5)$ has order $r = 5$:

$$G,\ 2G,\ 3G,\ 4G,\ 5G = O.$$

The public key $Q = (0, 2) = 4\,G$ hides $k = 4$.

Why the key is a hidden period

1. The key hides as a subgroup

With P of prime order r and $Q = [k]P$:

$$f(u,v) = [u]P + [v]Q = [u + kv]P.$$

f is constant exactly on the cosets of the hidden subgroup

$$L = \{(a,b) : a + kb \equiv 0 \pmod r\}$$
$$= \langle(-k, 1)\rangle.$$

So $f(u - k,\, v + 1) = f(u,v)$: a 2-D period; one more Q is k more copies of P.

2. Fourier sampling lands in $L^\perp$

Superpose all (u,v) and evaluate f into a point register:

$$\sum_{u,v} |u,v\rangle|O\rangle \xrightarrow{U_f} \sum_{u,v} |u,v\rangle\,|f(u,v)\rangle.$$

Inverse QFTs, measure $(s,t) \in L^\perp$:

$$L^\perp = \{(s,t) : -ks + t \equiv 0\},$$

$t \equiv ks, \quad k \equiv t\,s^{-1} \pmod r$ for any $s \neq 0$.

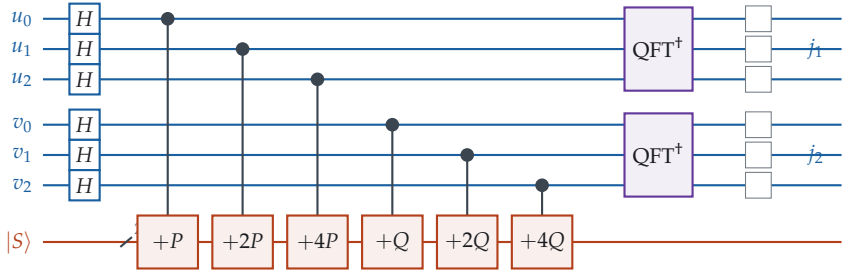
Toy curve (slide 70): $r = 5$, $k = 4$, so $L = \langle(1,1)\rangle$; a shot $(s,t) = (2,3)$ gives $k = 3 \cdot 2^{-1} = 3 \cdot 3 \equiv 4 \pmod 5$. Check $[k]P = Q$ classically. The oracle is the double-scalar multiplication $[u]P + [v]Q$; everything that follows makes it cheap.

Shor for ECDLP: what changes from factoring

	factoring N	**ECDLP,** $Q = [k]P$
secret	the order r of a mod N	the scalar k; the order r is public
hidden period	in one variable: $a^{x+r} = a^x$	in two: $f(u-k, v+1) = f(u,v)$
counting registers	one, $2n$ qubits	two, $n+1$ qubits each
controlled box	multiply by a^{2^j} mod N	add the point $[2^j]P$ or $[2^j]Q$
one box costs	modular additions	additions and a modular **division**
staying clean	multiply, swap, un-multiply	a second division clears λ
read-out	continued fractions, then a gcd	$k = t\,s^{-1}$ mod r; check $[k]P = Q$
128-bit security	$n = 3072$	$n = 256$

Same skeleton, a different box: a point addition costs a division, but the key is $12\times$ *shorter. Built here at about* 1 460 *qubits: ECDLP-256* 3.7×10^7 *Toffolis, RSA-2048* 7.0×10^9.

Two registers, one accumulator



$$\frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle\ |S+uP+vQ\rangle = \frac{1}{2^m}\sum_{u,v}|u\rangle|v\rangle\ |S+(u+kv)P\rangle$$

The accumulator depends on (u, v) only through $u + kv \bmod r$; two inverse QFTs turn that into (j_1, j_2). S is a fixed start point, since affine formulas cannot represent O (hit with probability $O(1/p)$).

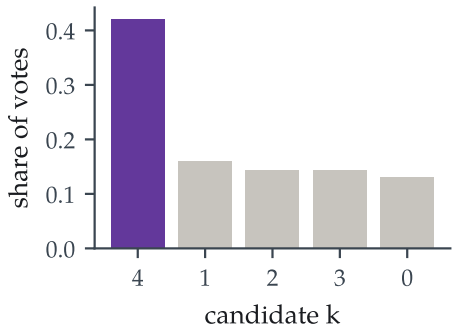
Post-processing: the key from (j_1, j_2)

A peak satisfies $j_2\, r/2^m \equiv k\,(j_1\, r/2^m) \pmod r$. Round, and divide:

$$a_i = \mathrm{round}\big(j_i\, r/2^m\big), \qquad k = a_2\, a_1^{-1} \bmod r.$$

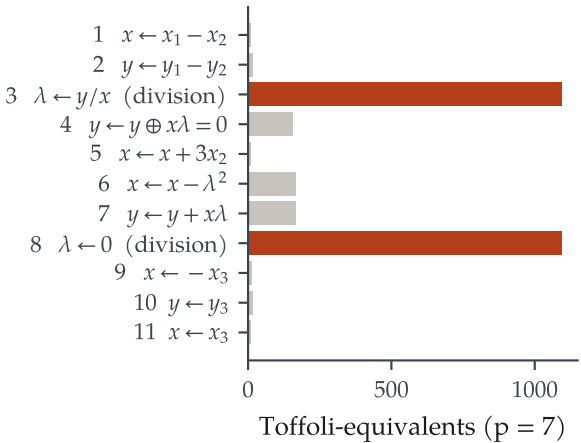
Every shot votes; the winner is checked with $kG = Q$.

Code: `ec_classical.ecdlp_postprocess` (backup). It also tries ± 1 around the rounding when 2^m is not a multiple of r (Ekerå).



> $k = 4$ *takes 42% of the vote, against 13–16% for each wrong key: a signal, not luck.*

One point addition is eleven reversible steps



In place, no garbage

Step 3 divides to get λ. Step 4 clears y, since $x\lambda = y$. Step 8 is the *same* division; by then $y/x = \lambda$ again, so it **clears** λ — uncomputation.

A division is Kaliski's binary extended-Euclid inversion: $2n$ rounds of shifts and subtractions.

> *The two divisions are 80% of the 2 744 Toffolis of one addition* ($p = 7$).

Making ECDLP fast: one optimisation per level

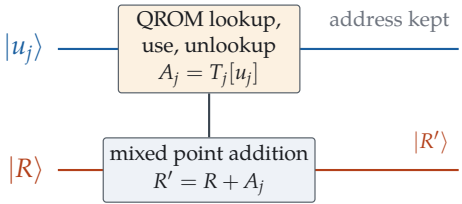
level	optimisation	measured on a fixed task
L0	temporary AND (Gidney)	2.50× fewer T over one inversion
L1	Gidney adder, copy-then-add control [106]	controlled add 128 → 63 Toffolis ($n = 32$)
L2	top-bit, pseudo-Mersenne reduction [1128]	doubling 25 → 4 Toffolis, 3.1% failures
L3	Montgomery on a carry-save tree [106]	depth ÷2.0 ($n = 16$)
L3	Kaliski: no counter, late reduction [106]	−29% Toffolis, −41% depth ($n = 12$)
L3	Euclidean dialog and replay [1128]	−54% Toffolis, −43% qubits ($p = 61$)
L4	Jacobian coordinates [106]	−45% Toffolis, $3n$ garbage qubits
L5	windows, zig-zag, one control qubit	514 → 34 additions at $n = 256$

[106] Kim et al., eprint 2026/106; [1128] Schrottenloher, eprint 2026/1128. Each variant is re-checked against the classical model inside the benchmark before it is counted; approximate ones record a failure rate.

L5: window by window — look up, add, unlook up

Cut each scalar into $\ell = \lceil n/w \rceil$ windows of w bits, $u = \sum_{j<\ell} u_j 2^{jw}$, and the oracle becomes a sum of table entries:

$$[u]P + [v]Q = \sum_{j<\ell} [u_j]P_j + \sum_{j<\ell} [v_j]Q_j, \qquad P_j = [2^{jw}]P, \quad Q_j = [2^{jw}]Q.$$



The table $T_j[i] = [i\,2^{jw}]P$ has 2^w entries, all computed classically before the run. The window is a coherent **address**, not a control bit: the circuit loads the point it selects, adds it, and erases it.

$$|z\rangle|0\rangle_A|R\rangle \xrightarrow{\text{lookup}} |z\rangle|T_j[z]\rangle_A|R\rangle \xrightarrow{\text{add}} |z\rangle|T_j[z]\rangle_A|R + T_j[z]\rangle \xrightarrow{\text{unlookup}} |z\rangle|0\rangle_A|R + T_j[z]\rangle$$

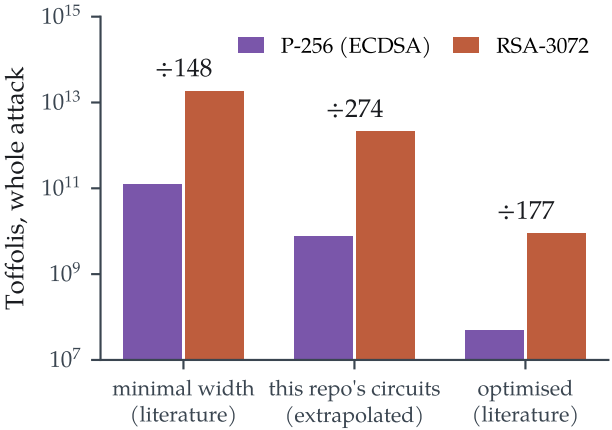
$n = 256$, $w = 16$: 16 windows per scalar, 32 such updates

Elliptic curves at toy size: what each level bought

level and technique	what it does	measured gain
L0 temporary AND	4-T ANDs in the inversion	T -60%
L1 Gidney adder	$n-1$ ANDs, control on a copy	controlled add -51%
L2 pseudo-Mersenne	subtract p: clear a bit, add f	doubling -84%
L3 carry-save Montgomery	one compressor tree	depth -50% ($4\times$ qubits)
L3 Kaliski, late reduction	reduce once, at the end	Toffolis -29%, depth -41%
L3 Euclidean dialog	2 bits per round, replayed	Toffolis -54%, qubits -43%
L4 Jacobian coordinates	no division at all	Toffolis -45% ($3n$ garbage)
L5 windows, zig-zag	16-bit windows	additions -93% ($514 \to 34$)

> *Halving a point addition is worth half as much as running fifteen times fewer of them.*

At 128-bit security



Toffolis

$(3072/256)^3 = 1\,728$, and $1\,728/6.7 \approx 260$: this repo's circuits $\approx 270\times$ fewer for P-256; minimal-width literature $148\times$; optimised literature $\approx 180\times$.

Qubits

depend on the construction: $2.6\times$ fewer in like-for-like circuits, but RSA-2048 now fits in $\approx 1\,400$ (Gidney 2025).

What it means

Elliptic curves are the cheaper target

The short key that makes ECC efficient classically makes it the easier quantum target: two orders of magnitude fewer Toffolis at the same security. Optimised: 4–9 $\times 10^7$ for a 256-bit curve (Google, IonQ 2026) against 6.5×10^9 for RSA-2048 (Gidney 2025).

Key exchange (ECDH)

Traffic recorded today can be decrypted later: *harvest now, decrypt later*.

Signatures (ECDSA)

Once a large machine exists, anyone can forge. Migrate to ML-KEM and ML-DSA (FIPS 203, 204).

> ***Resolved.*** *ECDSA falls first: same cubic cost, a 12× shorter key, a ≈ 7× heavier group operation — 20 minutes against 5 days on the same hardware.*

Part 5 — Where things stand in 2026

Choosing the code distance from the error budget

1. Budget. A run fails if any qubit fails in any round:

$$P_{\text{fail}} \approx P_L \times \text{qubits} \times \text{rounds}.$$

ECDLP-256: 1.4×10^{12} qubit-rounds; RSA-2048: 7×10^{13}. With $P_L = 10^{-15}$ they fail 0.1% and 7% of the time.

2. Scaling. $P_L(d) \approx 0.1\,(p/p_{\text{th}})^{(d+1)/2}$, $p_{\text{th}} \approx 1.2\%$: each $d \to d+2$ divides it by $\approx$12 at $p = 10^{-3}$.

3. Solve. $\dfrac{d+1}{2} \geq \dfrac{\log(0.1/P_L)}{\log(p_{\text{th}}/p)} = \dfrac{14}{1.09}$, so $d = 25$.

physical p	d	per logical
10^{-3}	25	1 352
5×10^{-4}	21	968
10^{-4}	13	392

Physical qubits per working logical qubit, $2(d+1)^2$, at $P_L = 10^{-15}$.

> *d grows with the log of the run; a* $10\times$ *better p cuts qubits per logical qubit* $3.5\times$.

From logical to physical qubits: the recipe

At $p = 10^{-3}$, a 1 μs cycle, 10 μs reaction:

1. Pick d with $0.1\,(p/0.0122)^{(d+1)/2} \le 10^{-15}$: $d = 25$.

2. Working logical qubit: $2(d+1)^2 = \mathbf{1\,352}$.

3. Idle logical qubit (yoked storage): $\approx$**430**.

4. Factories + workspace: **170 352**.

5. One CCZ every **25** μs: time = Toffolis $\times 25\,\mu$s.

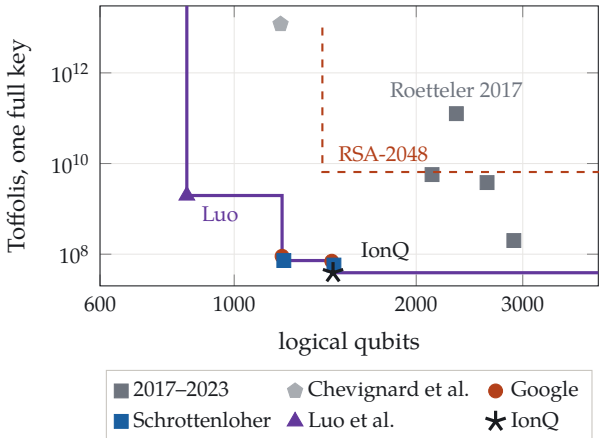
	ECDLP-256	RSA-2048
working logical	1 457	131
idle logical	–	1 280
Toffolis	3.68×10^7	6.5×10^9
physical	**2 140 216**	**897 864**
time	**15 min**	**4.96 d**

$1\,457 \times 1\,352 + 170\,352$; $1\,280 \times 430 + 131 \times 1\,352 + 170\,352$. RSA: 12.07 h $\times$ 9.2 shots = 4.63 d, $\div$ 93% error-free.

Gidney 2025's model (Sec. 3.2), in `shor_qiskit/physical.py`; it reproduces the paper's headline.

> *Physical* $\approx$ *logical* $\times\, 1\,352 + 170\,352$*; runtime* $\approx$ *Toffolis* $\times\, 25\,\mu s$.

The logical frontier, September 2026



ECDLP-256 frontier (purple)

Luo et al. $(835,\ 2.0 \times 10^9) \rightarrow$ Google $(1\,200,\ 9 \times 10^7) \rightarrow$ Schrottenloher $(1\,208,\ 7.2 \times 10^7) \rightarrow$ Google $(1\,450,\ 7 \times 10^7) \rightarrow$ IonQ $(1\,457,\ 3.9 \times 10^7)$.

RSA-2048 (dashed)

Gidney 2025 $(1\,399,\ 6.5 \times 10^9)$; Gidney–Ekerå $(6\,189,\ 2.7 \times 10^9)$. The 2026 progress on RSA was in the machines, not the arithmetic.

Act XII — from the papers as printed to below IonQ's count

Difficulty 12. Act VIII built eprints 2026/106 and 2026/1128 as printed: 2.6M Toffolis per point addition. IonQ reports 1.39M on 1 457 qubits. Where is the factor 2?

one windowed point addition, $n = 256$, $w = 16$	before	after
two in-place multiplications	3 910 552	1 441 130
square-subtract	1 050 623	79 160
table lookups	1 310 700	197 364
five modular additions	5 115	5 115
total	6 276 990	**1 722 769**
whole ECDLP-256 (34 → 28 additions)	213M	**48M**

3.6× *per addition from three changes: measured uncomputation, a cheaper GCD, a squarer.*

Act XII in plain words (1/3)

optimisation	what it does	what it bought
Measured lookup uncomputation	ten recomputed lookups become three loads and one shared phase repair	1.31M → 197 364 Toffolis per addition
Conditionally inverted GCD	keep both values odd, so each step is one uncontrolled adder between rows of CNOTs	in-place multiplication 1.96 → 0.72M
Squarer and fold	squaring as unconditional signed subtractions, reduced with the shape of p	square-subtract 1.05M → 79 160
Carry-is-flag adder (IonQ)	add with carry-out; the carry decides the reduction and is cleared by a measurement and a phase fix	replay step 712 → 352; addition 1.71 → 1.44M
37 careful steps, own replay	the one sum the carry misses ($x + y = p$) occurs early; multiplication gets its own replay	exact on every test; 569 600 → 307 574

All at $n = 256$, $w = 16$, per windowed point addition unless stated.

Act XII in plain words (2/3)

optimisation	what it does	what it bought
Signed windows	read each window as $\pm d$, look up $\lvert d\rvert$, negate y for the sign	2^{15}-entry tables, no O; 4.6% below IonQ
Peak chasing	free the scratch of whichever step sets the peak; pack the record, share registers	2 233 $\rightarrow$ 1 457 qubits, no extra Toffoli
SELECT-SWAP	load 2^k words at once and route one with Fredkins, on qubits idle at that moment	1.327 $\rightarrow$ 1.311M executed at 1 457 qubits
Lean cells	CDKM adders, a 1-ancilla comparator, chunked tests, a constant adder with a borrowed increment	2 251 $\rightarrow$ 1 246 qubits, 1.71 $\rightarrow$ 2.82M Toffolis
Controlled CDKM at $3n$	carries computed uncontrolled; only the sum write is controlled	$4n \rightarrow 3n$ Toffolis

secp256k1 at full size: what each change bought

one point addition, $n = 256$, $w = 16$	before $\rightarrow$ after	gain
Measured lookup clean-up	lookups 1.31M $\rightarrow$ 197k Toffolis	-85%
Inverted-step GCD (IonQ)	multiplication 1.96 $\rightarrow$ 0.72M	-63%
Squarer and fold	square-subtract 1.05M $\rightarrow$ 79k	-92%
Three changes together	**6.28 $\rightarrow$ 1.72M Toffolis**	-73%
Carry-is-flag adder (IonQ)	replay step 712 $\rightarrow$ 352	-51%
Signed windows	tables $2^{16} \rightarrow 2^{15}$ entries	-7%
Peak chasing and sharing	qubits 2 233 $\rightarrow$ 1 457	-35%
SELECT-SWAP, $3x$ lookup	1.327 $\rightarrow$ 1.311M executed	-1.2%

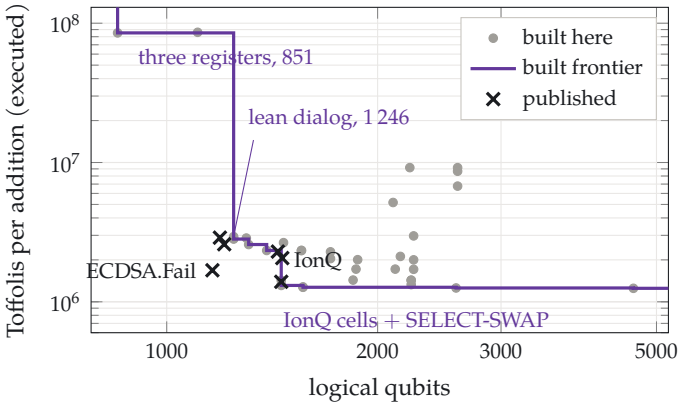
> 1.31*M executed Toffolis per addition on* 1 457 *qubits*, 5.9% *below IonQ. Whole ECDLP-256:* 213*M* $\rightarrow$ **37.1*M*** (-83%).

Other trade-offs: fewer qubits, or less depth

one point addition, $n = 256$	gain	price
Lean cells: CDKM adders, a 1-ancilla comparator, a borrowed increment	qubits 2 251 $\to$ 1 246 (-45%)	Toffolis 1.71 $\to$ 2.82M ($+65\%$)
Register-shared Euclid (Luo et al.): no record kept	qubits 1 246 $\to$ 1 107	30$\times$ the Toffolis
Three field registers: measure y away, repair the phase	851 qubits	85.2M Toffolis per addition
Carry-lookahead adders, controls fanned out	Toffoli depth 1.50 $\to$ 0.39M (-74%)	4.6$\times$ the Toffolis

> *The headline circuit sits at* 1 457 *qubits; going narrower or shallower is possible, and it costs Toffolis.*

The built frontier



Grey: every circuit built at $n = 256$ (secp256k1's prime); Qualtran counts the same Toffolis. Crosses: [1128], Google, IonQ, ECDSA.Fail.

The frontier ends at IonQ's cells with SELECT-SWAP: 1 457 qubits, 1.31M executed — below IonQ's own point.

> ***Resolved.*** *The factor 2 between [1128] and IonQ is three ideas, all reproduced.*

What the optimisations in the repo bought

	biggest single gains	overall
Factoring	ripple-carry adders: T −95 to −99% one counting qubit: qubits −41 to −50% measured clean-up of a lookup: −96%	T −96% at $N = 15$
Elliptic curves, toy size	Euclidean dialog: Toffolis −54%, qubits −43% pseudo-Mersenne doubling: −84% 16-bit windows: point additions −93%	each level measured on a fixed task
secp256k1, $n = 256$	three changes: 6.28 → 1.72M per addition (−73%) peak chasing: 2 233 → 1 457 qubits (−35%) 1.31M per addition, 5.9% below IonQ	whole ECDLP-256: 213 → 37.1M (−83%)

> *Every number is a count the repo measured on circuits it built, each optimisation switched on and off.*

Seven things to keep

1. **Shor is phase estimation of a group operation.** The quantum part is short; the cost is reversible arithmetic.
2. **Uncomputation is mandatory.** Garbage that records x erases the peaks.
3. **The price is T gates.** A Toffoli costs 7; a small rotation $\approx 3\log_2(1/\varepsilon)$.
4. **Optimise in order.** Remove rotations, then the counting register, then additions.
5. **Elliptic curves fall first.** Same $\Theta(n^3)$, a $12\times$ shorter key: 150–270$\times$ cheaper.
6. **Gates are Hamiltonians.** Universality is a Lie-bracket condition; every platform reaches the CNOT point.
7. **The frontier moved in 2026.** ECDLP-256 on $\approx 10^3$ logical qubits, 4–9 $\times\ 10^7$ Toffolis: 5×10^5 superconducting qubits for minutes, or 2×10^4 ions or atoms for weeks.

Optimisations in detail — factoring (Act VII)

Act VII — what each optimisation removes

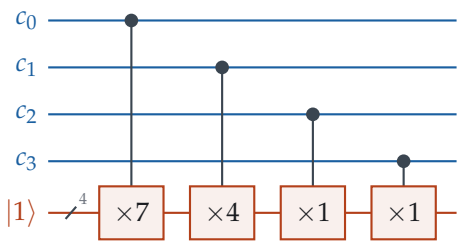
optimisation	what it removes	measured effect
precomputed powers	$2^t - 1$ rung applications	t rungs instead
control placement	controls that cancel anyway	12.8× fewer CNOTs per adder
semiclassical QFT	the counting register	$4n + 2 \to 2n + 3$ qubits
ripple-carry adder	every rotation in the arithmetic	T ÷19 ($n = 4$) to ÷99 ($n = 12$)
controlled MAJ/UMA	4 of 6 controlled gates per bit	$2n$ controlled gates, not $6n$
temporary AND	the T cost of uncomputing	3.5× fewer T per lookup
windowing	most modular additions	$n \to \lceil n/w \rceil$; depth ÷3.4 ($n = 8$)
unary iteration	the lookup's 2^w ancillas	59 → 31 qubits ($n = 8$, $w = 4$)
measured uncomputation	the second lookup	26× cheaper at $w = 11$
nested windows	exponent rungs and their controls	8 → 4 multiplications ($N = 15$)
coset representation	the modular reduction	3–3.9× fewer Toffolis
approximate QFT	the smallest rotations	−34% rotations ($n = 8$)

All twelve are built and checked in `shor_qiskit/`*; numbers from* `shor-complete.tex`*, Part V.*

Free, before anything else: the powers collapse

Rung j needs $U_a^{2^j}$, and that is **one** multiplication by a classical constant:

$$U_a^{2^j} = U_{a^{2^j} \bmod N} \qquad \text{(repeated squaring, on a laptop).}$$



$N = 15, a = 7$: rungs $\times 7, \times 4, \times 1, \times 1$

Without it: $2^t - 1$ applications of U_a — 15 instead of 4 at $t = 4$, exponentially many at size.

At $N = 15$ the rungs from the third on multiply by 1; a circuit compiled for 15 deletes them (7 NMR qubits, 2001).

```
for i in range(t):                          # the ladder: t rungs
    c_ua(qc, ctr[i], pow(A, 2**i, N), tgt, anc, sf[0], sf[1], N)
```

Free, before anything else: control only what does not cancel

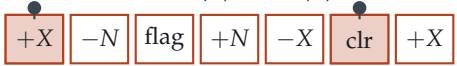
Control only some gates of $V = G_L \cdots G_1$: with the control off, the rest still runs. So

C-V needs a control on G_j only if the uncontrolled residue would not cancel.

Beauregard: blocks (1), (5), (7)



this code: block (1), and (6) as a Toffoli



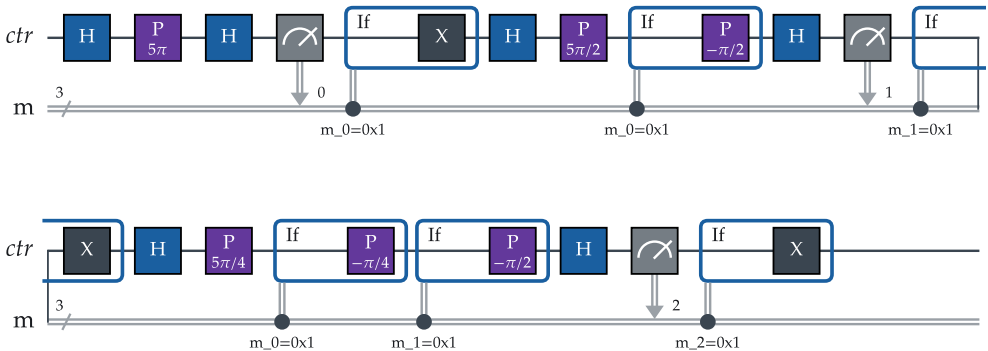
Conjugation: in $ABA^\dagger$ control B only — Draper's adder controls its phases, not its QFTs.

Compensating residue: with the control off, blocks (2)–(4) and (6) cancel each other, because $y < N$.

Measured, $N = 15$, $a = 7$ (CNOTs / depth)

modular adder 2 634 / 4 836 → **206** / **354** (12.8×); whole C-U_a 20 892 / 37 727 → **1 748** / **2 957** (12.0×)

In Qiskit: a dynamic circuit



The loop on its own, with phase gates as the rungs ($y = 5, t = 3$): Aer reads back `101` on every shot.

`if_test`: **mid-circuit measurement and feed-forward**.

The loop in code

```
def semiclassical_iqft(qc, ctrl, out, rungs):
    t = len(rungs)
    assert len(out) >= t, "one classical bit per rung"
    for i in range(t):
        qc.h(ctrl)
        rungs[t - 1 - i](qc, ctrl)                  # heaviest rung first
        for j in range(i):                          # the inverse QFT, classically
            with qc.if_test((out[j], 1)):
                qc.p(-math.pi / 2 ** (i - j), ctrl)
        qc.h(ctrl)
        qc.measure(ctrl, out[i])
        with qc.if_test((out[i], 1)):               # reset for the next rung
            qc.x(ctrl)
```

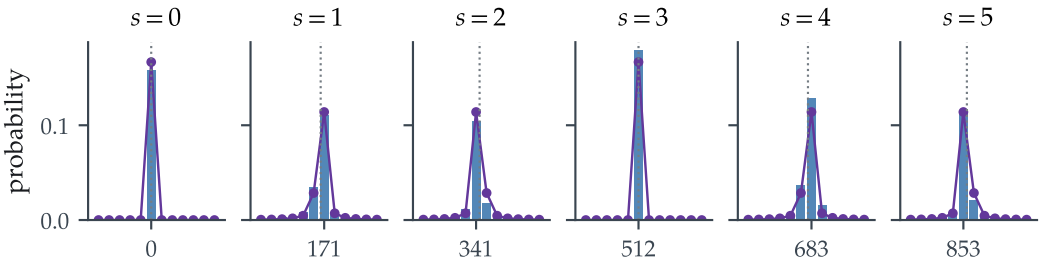
One loop, both problems

The rungs are callbacks: the same function drives factoring and ECDLP.

Checked exactly

Enumerating the measurement branches, its distribution equals the full circuit's to 10^{-13}.

$N = 21$ on 13 qubits instead of 22



bars: Aer, 1024 shots of the one-counting-qubit circuit; dots: the exact distribution around each peak $s \cdot 2^{10}/6$.

Same distribution

total variation distance 0.075, within the shot-noise bound 0.108.

Same success

one shot gives $r = 6$ with probability 0.322 exact, 0.313 measured;
$21 = 7 \times 3$.

Controlling the ripple-carry adder: two gates of six



Only the two CNOTs into b (vermillion) take the outer control. With it off, MAJ leaves two gates and UMA the same two reversed, so the chains cancel from the inside out.

$2n$ controlled gates, not $6n$.

```
def _maj(qc, c, b, a, ctrls=()):
    qc.mcx([*ctrls, a], b)              # plain CNOT when ctrls is empty
    qc.cx(a, c)
    qc.ccx(c, b, a)

def _uma(qc, c, b, a, ctrls=()):
    qc.ccx(c, b, a)
    qc.cx(a, c)
    qc.mcx([*ctrls, c], b)              # plain CNOT when ctrls is empty
```

Ripple-carry against Fourier, measured

adder		qubits	rot.	Toffoli	depth
$N = 15$	Fourier	7	125	0	492
	ripple-carry	13	**0**	71	578
$N = 63$	Fourier	9	245	0	948
	ripple-carry	17	**0**	99	**844**

One modular adder. Rotations grow as n^2 ($125 \to 245$), Toffolis as n ($71 \to 99$); from $n = 6$ ripple-carry is shallower too.

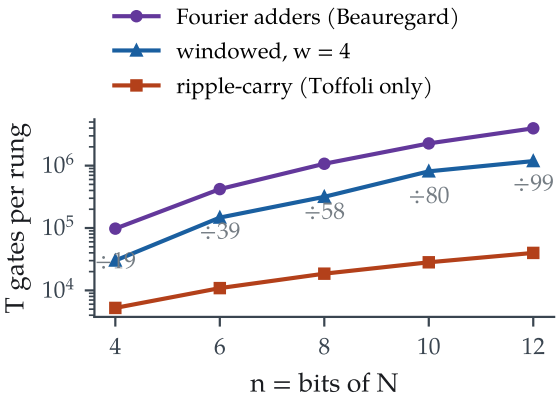
Which to use

Simulating small N: Fourier — $4n + 2$ qubits against $5n + 4$ is the whole cost of a statevector.

Fault tolerance: ripple-carry. No rotations, and the T count is the bill.

> *The smallest Fourier rotation is* $2\pi/2^{n+1}$*:* 10^{-616} *rad at* $n = 2048$*. At size, Fourier arithmetic must be approximated before it can be run at all.*

Toffolis are cheaper than rotations



One rung, T gates with every rotation synthesised.

Ripple-carry: 7–8× **more** Toffolis, but no rotations.
T: ÷19 at $n = 4$, ÷99 at $n = 12$.

The gap grows like n: rotations scale as n^4, Toffolis as n^3.

The temporary AND, in code

```
def and_compute(qc, a, b, t):
    qc.h(t)
    qc.t(t)
    qc.cx(b, t)
    qc.tdg(t)
    qc.cx(a, t)
    qc.t(t)
    qc.cx(b, t)
    qc.tdg(t)
    qc.h(t)
    qc.sdg(t)
```

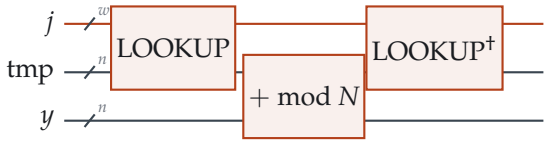
bits	L	T (Toffolis)	T (temp. ANDs)	saving
2	4	42	12	3.50×
3	8	98	28	3.50×
4	16	210	60	3.50×
5	32	434	124	3.50×

one unary-iteration lookup, transpiled to Clifford+T

Exactly 7/2: half the Toffolis were uncomputes, now free; the rest fell from 7 T to 4. The last gate is $S^\dagger$: three plausible variants differ only there, and all are wrong.

```
def and_uncompute(qc, a, b, t, cbit):
    qc.h(t)
    qc.measure(t, cbit)
    with qc.if_test((cbit, 1)):
        qc.cz(a, b)
        qc.x(t)                                  # return the ancilla to |0>
```

Windowing: look the answer up, then add once



tmp $\leftarrow T[j]$; $y \leftarrow y +$ tmp; tmp $\leftarrow 0$.

The addend is now *quantum*: Draper's phase ramp takes its rotations from the bits of tmp; the seven blocks are unchanged.

```
def add_quantum_mod(qc, t, y, sign, flag, N):
    yw = list(y) + [sign]
    add_quantum(qc, t, yw, +1)            # (1) + t
    add_const(qc, -N, yw)                 # (2) - N
    qc.cx(sign, flag)                     # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)        # (4) + N  if flag
    add_quantum(qc, t, yw, -1)            # (5) - t
    qc.cx(sign, flag)                     # (6) [R < t] == not flag, so this
    qc.x(flag)                            #     pair returns flag to |0>
    add_quantum(qc, t, yw, +1)            # (7) + t
```

Windowing, in code

```python
def windowed_c_mult_acc(qc, c, k, x, y, sign, flag, N, t, anc, w):
    n = len(y)
    for i in range(0, len(x), w):
        win = list(x[i:i + w])
        # window value j contributes (k * j * 2^i) mod N
        table = [((k * j) << i) % N for j in range(2**len(win))]
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])
        add_quantum_mod(qc, t[:n], y, sign, flag, N)
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])   # uncompute
```

Why $c = 0$ entries of 0 are the trick

With $c = 0$ the lookup writes 0, and the seven blocks with $t = 0$ undo themselves: no controlled gate in the adder.

The second `lookup_ui` uncomputes the first — a cost the next slides remove.

Windowing, measured

$y \mathrel{+}= kx$, $N = 255$	adds	qubits	CNOT	depth
shift-and-add	8	19	7 390	13 020
window $w = 2$	4	29	4 840	6 497
window $w = 3$	3	30	3 902	4 873
window $w = 4$	2	31	**3 264**	**3 866**

full circuit, $N = 15$	qubits	CNOT	depth
plain, $t = 8$	18	20 462	34 537
windowed $w = 2$, $t = 8$	24	14 276	**17 028**

Depth falls faster than additions

Lookups are shallow; the additions they replace were not. At $n = 8$: CNOTs $\div 2.3$, depth $\div 3.4$, for 12 more qubits.

End to end on Aer

$N = 15$, $t = 8$, 24 qubits: all 2 048 shots on the four peaks.
$N = 21$, $t = 10$, 29 qubits (68 min): the six peaks, $r = 6$, $21 = 7 \times 3$.

Unary iteration, in code: m ancillas, the control at the root

```
def _walk(qc, ctrl, addr, out, table, offset, anc):
    if not addr:
        T = table[offset] if offset < len(table) else 0
        for b in range(len(out)):
            if (T >> b) & 1:
                qc.cx(ctrl, out[b])           # leaf: data load is CNOTs
        return
    a, rest = addr[-1], addr[:-1]             # split on the most significant bit
    half, node = 1 << len(rest), anc[0]
    qc.ccx(ctrl, a, node)                     # node = ctrl AND a      (upper half)
    _walk(qc, node, rest, out, table, offset + half, anc[1:])
    qc.cx(ctrl, node)                         # node = ctrl AND NOT a  (lower half)
    _walk(qc, node, rest, out, table, offset, anc[1:])
    qc.cx(ctrl, node)                         # put node back to ctrl AND a ...
    qc.ccx(ctrl, a, node)                     # ... and clear it

def lookup_ui(qc, ctrl, addr, out, table, anc):
    assert len(anc) >= len(addr), "unary iteration needs one ancilla per level"
    _walk(qc, ctrl, list(addr), list(out), list(table), 0, list(anc))
```

Measured uncomputation, in code: the controller sits in the middle

```
def phase_fixup(qc, addr, F, one, unary_lo, anc_hi, l):
    from windowed import _unary_gate
    lo, hi = list(addr[:l]), list(addr[l:])
    g = _unary_gate(l)
    qc.append(g, lo + list(unary_lo[:1 << l]))
    qc.x(one)                                          # always-true control
    _walk_cz(qc, one, hi, list(unary_lo[:1 << l]), F, 0, list(anc_hi))
    qc.x(one)
    qc.append(g.inverse(), lo + list(unary_lo[:1 << l]))
```

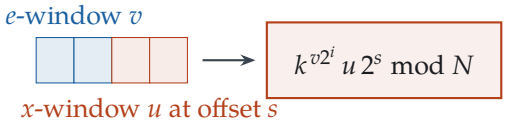
Not one static circuit

F depends on the outcome, so it runs as circuit → measure → classical → circuit, as on hardware (`run_windowed_acc_mbu`).

Checked

Exactly $\mathrm{diag}((-1)^{F[a]})$ for every split; in the windowed accumulate, 180 mid-circuit measurements leave every register exact.

Nested windows: window the exponent too



- $\lceil t/w_e \rceil$ multiplications instead of t;
- **no control**: window 0 selects $k^0 = 1$;
- the swap becomes a **relabelling**: no Fredkins.

$N = 15, t = 8$	mults	CNOT	depth
plain	8	20 462	34 537
windowed $w_m = 2$	8	14 276	17 028
nested $w_e = 2$	4	**11 908**	**14 229**
nested $w_e = 4$	2	16 292	20 095

Windows have an optimum

At $w_e = 4$ the 2^6-entry table costs more than the two multiplications it saves.

Nested windows, in code

```
for i in range(0, len(e), we):
    ei = list(e[i:i + we])
    ne = len(ei)
    kes = [pow(k, (2**i) * v, N) for v in range(2**ne)]
    ...
    for j in range(0, n, wm):
        mi = A[j:j + wm]
        addr = ei + list(mi)
        table = [((kes[ad & ((1 << ne) - 1)] * (ad >> ne)) << j) % N
                 for ad in range(1 << len(addr))]
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
        qc.append(madd, tmp[:n] + B + [sign, flag])
        lookup_ui(qc, one, addr, tmp[:n], table, anc[:len(addr)])
    ...                                     # A -= B * k_e^-1, the same way
    A, B = B, A                             # relabelling swap -- free
```

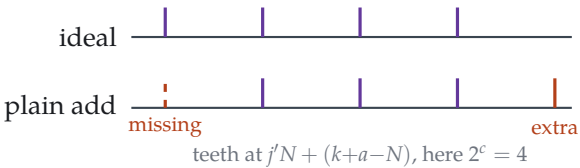
Checked: $|e\rangle|1\rangle|0\rangle \mapsto |e\rangle|7^e \bmod 15\rangle|0\rangle$ for all $e < 16$; factors 15 on 23 qubits, 2 multiplications.

Not with one counting qubit: w_e exponent bits must coexist as an address; the two are ends of one spectrum.

Coset representation: delete the modular reduction

Store k as a comb over c padding qubits (Zalka; Gidney–Ekerå):

$$|k\rangle_{\text{coset}} = 2^{-c/2} \sum_{j<2^c} |jN + k\rangle$$



Every branch is $\equiv k \pmod N$, so a **plain** addition of a gives branches $\equiv k + a$. When $k + a \geq N$ the comb re-indexes, and differs from the ideal state in **two edge teeth** out of 2^c. That is the entire approximation.

```
def add(qc, X, reg, N=None):
    if N is not None:
        X %= N
    add_const(qc, X, list(reg))
```

One addition, coset vs seven blocks

Fourier: 1.7× ($N = 15$) to 2.8× ($N = 1023$) fewer CNOTs. Ripple-carry: 3.0× to 3.9× fewer Toffolis.

Coset representation, measured

N	c	error, 8 adds	bound $8/2^c$
15	2	0.749	2.000
	3	0.373	1.000
	4	0.129	0.500
	5	0.031	0.250
	6	0.000	0.125
31	2	0.507	2.000
	4	0.061	0.500

The error halves per padding qubit. Approximate by design: the tests measure a rate, not equality.

A friendly modulus hides the effect

At $N = 21$ the error was 0 for every c: $2^n - N = 11$ leaves so much headroom that no branch wraps. Hence 15 and 31, just below 2^n.

Multiplying works too

Branch bits differ, yet $\sum_i v_i (A 2^i \bmod N) \equiv A(jN + x) \equiv Ax$. At $N = 7$, $c = 4$: worst error 0.0135.

The coset order-finding circuit (`coset.order_circuit_coset`) is checked at $N = 15$.

Approximate QFT: fewer rotations, but errors add up

Drop every $\mathrm{CP}(\pi/2^k)$ with $k > d$, in every adder's QFT. On $N = 15$:

QFTs	small rotations	distance from exact	P(one shot gives r)
exact	12 855	0.006	0.501
$d = 4$	12 837	0.006	0.501
$d = 3$	10 905	**0.138**	0.451
$d = 2$	7 050	**0.594**	0.245

320 Fourier additions, each with its own approximation error: they accumulate. Shot-noise bound: 0.032.

> *One error budget is shared by every approximation, so d must grow like $\log_2(\#\textit{additions}/\varepsilon)$. At $n = 8$, $d = 4$ already removes a third of the rotations.*

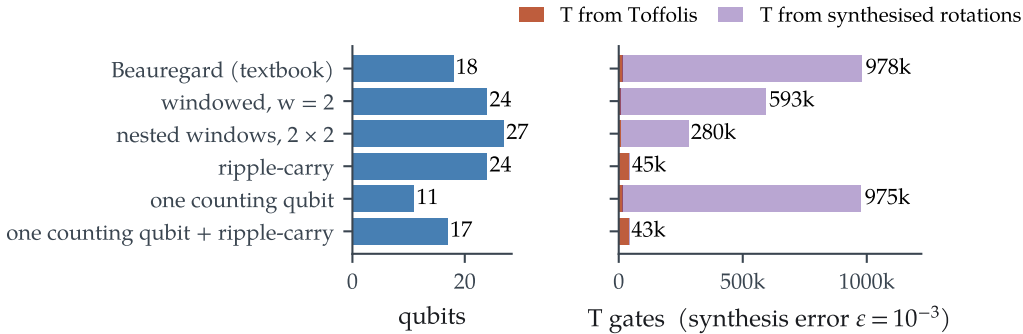
Act VII in plain words (1/2)

optimisation	what it does	what it bought
Precomputed powers	$U_a^{2^j}$ is one multiplication by the constant $a^{2^j} \bmod N$, squared on a laptop	t rungs instead of $2^t - 1$ multiplications
Control placement	of the seven blocks of a controlled adder, only those that do not cancel need the control	$12.8\times$ fewer CNOTs per adder ($N = 15$)
Semiclassical QFT	measure each counting qubit first, apply the rotations classically, reuse one qubit	$4n + 2 \to 2n + 3$ qubits; $N = 21$ on 13
Ripple-carry adder	add with Toffolis and CNOTs, no small rotations	T $\div 19$ ($n = 4$) to $\div 99$ ($n = 12$)
Controlled MAJ/UMA	control only the two CNOTs per bit that write the target	$2n$ controlled gates, not $6n$
Temporary AND	a Toffoli onto $\lvert 0\rangle$ costs 4 T; undo it by an X measurement and a classically controlled CZ	14 T $\to$ 4 per pair; lookups $3.5\times$ cheaper

Act VII in plain words (2/2)

optimisation	what it does	what it bought
Windowing	take w bits at once, look up their contribution, add once; the control becomes one more address bit	$n \to \lceil n/w \rceil$ additions; one rung $4.0 \to 1.2$M T
Unary iteration	walk the table's addresses with a tree of ANDs on about w ancillas	$59 \to 31$ qubits ($n = 8$, $w = 4$)
Measured uncomputation	erase a lookup by an X measurement and an $O(\sqrt{L})$ phase repair, not a second lookup	$4\,094 \to 157$ Toffolis at $w = 11$
Nested windows	window the exponent too: one multiplication covers several exponent bits	$8 \to 4$ multiplications ($N = 15$)
Coset representation	store k as $\sum_j \lvert jN + k\rangle$; a plain adder then adds mod N, error $\approx 2^{-c}$	3.0–3.9$\times$ fewer Toffolis per addition
Approximate QFT	drop the rotations finer than $\pi/2^d$	-34% rotations; errors add up

All together: $N = 15$



Priced with synthesis, ripple-carry wins by 20×: 17 qubits and 42 600 T, against 18 and 978 000.

> ***Resolved.*** *Remove rotations first, then qubits, then additions.*

Optimisations in detail — elliptic curves, toy size (Act VIII)

L0: temporary ANDs — and which Toffolis are counted

n	ANDs	AND†s	T (Toffoli)	T (AND)
5	912	892	14 938	5 958
8	2 229	2 197	36 694	14 628
12	4 785	4 737	79 254	**31 740**

one modular inversion

A flat 2.50×: ANDs and AND†s come in equal numbers, so the average is $(4+0)/2 = 2$ T against 7 — 3.5×, diluted by the Fredkins and dirty-target Toffolis that cannot use the trick.

What counts as a Toffoli

Both papers count CCX, CCZ and AND gates; a measured AND† has no T and is free. Counting every three-qubit operation instead nearly doubles Gidney-style circuits.

`ec_cost` reports both (`toffoli_paper`, `toffoli_equiv`). Quoting the wrong one halves every optimisation here.

L1: which adder, and how to control it

	add		controlled add		
n	CDKM	Gidney	promote	copy, CDKM	copy, Gidney
8	16	7	32	24	15
16	32	15	64	48	31
32	64	31	128	96	**63**

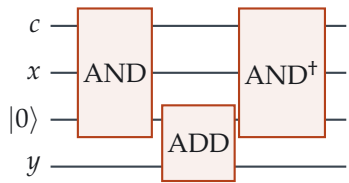
Toffolis; "promote": every CNOT of the adder becomes a Toffoli

CDKM: $2n$ Toffolis on $2n + 1$ qubits.

Gidney: $n - 1$ ANDs on clean targets, free to uncompute (measure and fix the phase), on $3n - 1$ qubits.

Copy-then-add: AND the addend into a clean register under the control, add unconditionally, un-AND. $2.0\times$ fewer Toffolis than promoting, of which $1.33\times$ is the control style alone.

Copy-then-add, in circuit and in code



$y \mathrel{+}= c \cdot x$; the copy is $|0\rangle$ again

```
def cadd(ctx, ctrl, x, y, copy, anc):
    assert len(copy) == len(x)
    for i, q in enumerate(x):
        ctx.and_(ctrl, q, copy[i])
    add(ctx, copy, y, anc)
    for i, q in enumerate(x):
        ctx.and_dg(ctrl, q, copy[i])
```

Only the copy sees the control: n ANDs in, n free AND†s out, and the adder in the middle is the cheap uncontrolled one. The same pattern controls every adder in the ECDLP circuit.

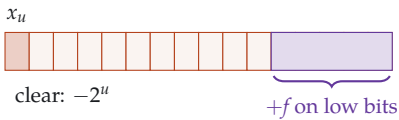
L2: approximate and pseudo-Mersenne reduction

Compare only the top bits

"At least p?" is right unless the top bits tie, probability $\approx 2^{-\text{msbs}}$, and the flag comes *before* the subtraction: one controlled subtraction instead of two steps.

Use the shape of the prime

secp256k1: $p = 2^u - f, f = 2^{32} + 977$. "At least p?" is almost "bit u set?", and subtracting p is "clear bit u, add f" — on the low bits only.



$p = 4093$	doubling		ctrl. add	
	Toff.	fail	Toff.	fail
exact	25	0%	73	0%
top 4 bits	17	3.1%	51	9.7%
pseudo-M.	**4**	3.1%	49	1.0%

Failures track $2^{-\text{msbs}}$: 8 bits give 0.17%.

Pseudo-Mersenne doubling, in code — and why failures compound

```
def moddbl_pm(m, x, q, lsbs=None):
    ...
    u, f = pm
    ctx, n = m.ctx, len(x)
    ...
    z = m.anc(1, "z")
    xe = A.shift_up(x, z[0])                        # n+1 bits; xe[n] is the overflow
    cp, sc = m.anc(lsbs, "cp"), m.anc(lsbs, "sc")
    A.cadd_const(ctx, xe[n], Reg(list(xe[:lsbs])), f, cp, sc)
    m.free(cp, sc)
    ctx.cx(xe[0], xe[n])                            # clears the bit == subtracts 2^u
    for i in range(n - 1, -1, -1):
        ctx.swap(xe[i], xe[i + 1])
```

Nothing compares anything: clearing the overflow bit *is* the subtraction of 2^u. A 2-bit comparison failing 12% per step makes a whole multiplier fail 81% ($p = 31$): hence 40–50 bits at $n = 256$.

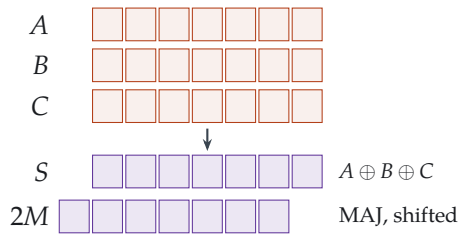
One case is not rare: Alg. 10 cannot reduce $x + y = p$, and every multiplication meets it once. Alg. 11 catches it, with its flag test gated on the control.

L3: Montgomery multiplication over a carry-save tree

[106] rewrites the reduction $c \mathrel{+}= Mp$ as $c \mathrel{+}= c_w \cdot d$ with a **compile-time** $d = (p'_w \bmod 2^w)\, p$: the shape of the multiplication step, so one carry-save tree serves both.

A 3:2 compressor makes three addends two, $A + B + C = S + 2M$, with one AND per bit and no carry chain: $O(\log w)$ depth.

A depth optimisation: $\div 2.0$ at $n = 16$, for $4\times$ the qubits; it does not beat the QROM multiplier on gates.



$n = 16, w = 4$	Toffoli	qubits	depth
schoolbook	2 080	86	13 043
carry-save tree	1 566	324	**6 669**
QROM lookup	1 054	347	4 780

The 3:2 compressor, in code

```
def csa_compress(m, A, B, C):
    ctx, W = m.ctx, len(A)
    assert len(B) == len(C) == W
    S, M = m.anc(W, "csaS"), m.anc(W, "csaM")
    for i in range(W):                      # S = A ^ B ^ C
        ctx.cx(A[i], S[i])
        ctx.cx(B[i], S[i])
        ctx.cx(C[i], S[i])
    for i in range(W - 1):                  # M = MAJ, written one place up
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
        ctx.and_(B[i], C[i], M[i + 1])
        ctx.cx(A[i], M[i + 1])
        ctx.cx(A[i], B[i])
        ctx.cx(A[i], C[i])
    return S, M
```

The rewritten reduction over-adds by up to $2^{2w}p$, so the accumulator settles at $n + 2w$ bits, not Montgomery's $n + 2$; the tests measure that width rather than trust it.

L3: Kaliski with every round run, and the reductions postponed

Run every round. An inert round only doubles r, so every input ends at $-x^{-1}2^{2n}$: no counter, no corrective halvings — one negation instead.

Reduce once, at the end. With r, s on $2n$ bits a modular doubling is a shift, i.e. a renaming of wires, and a modular addition a plain one.

	Toffoli		depth	
n	before	after	before	after
5	1 242	817	4 846	2 664
8	3 045	2 077	12 271	7 019
12	6 585	**4 653**	27 239	**16 059**

Häner et al. 2020 → [106], +11% qubits

```
cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
A.csub(ctx, b[0], u, v, cp, sc)              # v -= u
m.free(cp, sc)
cp, sc = m.anc(W, "cp"), m.anc(W, "sc")
A.cadd(ctx, b[0], r, s, cp, sc)              # s += r   (plain, not mod p)
m.free(cp, sc)
_rot_down(ctx, v)                            # v /= 2
_rot_up(ctx, r)                              # r *= 2   (free: no reduction)
```

L3 rebuilt: the Euclidean dialog

Inversion and multiplication are **one job** ([1128], after Khattar et al.). Run the binary GCD on $(u, v) = (p, x)$ *without* the Bézout coefficients; keep only two decision bits per round, the **dialog**:

repeat $\approx 1.413n$ times:
  $b_0 := v \bmod 2, \quad b_1 := [u > v]$
  record b_0 and $b_0 b_1$
  if $b_0 b_1$: swap u, v
  if b_0: $v \mathrel{-}= u$
  $v \mathrel{/}= 2$

Why $1.413n$ rounds

Halving always removes a bit, the subtraction a second one half the time: uv shrinks by $3/8$ per round, $2n/\log_2(8/3) \approx 1.413n$. Measured: $1.383n$ at $n = 32$, $1.409n$ at $n = 256$; spread $\approx 0.6\sqrt{n}$, hence the $+c\sqrt{n}$ margin.

Packing the record

$(b_0, b_0 b_1)$ is never $(0, 1)$: three pairs have $27 < 32$ states, so they fit in 5 bits. Record $2.83n \to 2.355n$, at a fixed size.

One round of the dialog, in code

```
def dialog_round(m, u, v, rec):
    ctx, n = m.ctx, len(u)
    b0, b0b1 = rec
    ctx.cx(v[0], b0[0])                          # b0 = v mod 2

    t, sc = m.anc(1, "t"), m.anc(n, "sc")
    A.gt_uint(ctx, u, v, t[0], sc)               # t = [u > v]
    ctx.and_(b0[0], t[0], b0b1[0])               # b0b1 = b0 AND b1
    A.gt_uint(ctx, u, v, t[0], sc)
    m.free(t, sc)

    for i in range(n):
        ctx.cswap(b0b1[0], u[i], v[i])           # if b0b1: swap u, v

    cp, sc = m.anc(n, "cp"), m.anc(n, "sc")
    A.csub(ctx, b0[0], u, v, cp, sc)             # if b0: v -= u
    m.free(cp, sc)

    for i in range(n - 1):                       # v /= 2 (v is even now)
        ctx.swap(v[i], v[i + 1])
```

The replay: the same decisions, applied to any pair

The updates the full extended Euclid makes to (r, s) are **linear** in the recorded bits, so they can be replayed afterwards onto any pair:

$$s \leftarrow 2s; \quad \text{if } b_0 : \; s \mathrel{+}= r; \quad \text{if } b_0 b_1 : \; \text{swap}(r, s).$$

Direction matters

Read backwards from $(y, 0)$ the replay ends at $(0, xy \bmod p)$ and needs only doubling; forwards it divides, with halving. Run the circuit in reverse to divide.

```
def bezout_replay(m, r, s, recs, q, dbl=None, cadd=None):
    dbl = dbl or (lambda mm, reg: MA.moddbl(mm, reg, q))
    cadd = cadd or (lambda mm, c, a, b: MA.cmodadd(mm, c, a, b, q))
    for b0, b0b1 in reversed(recs):
        dbl(m, s)                                   # s <- 2s mod q
        cadd(m, b0[0], r, s)                        # if b0: s += r mod q
        for i in range(len(r)):
            m.ctx.cswap(b0b1[0], r[i], s[i])        # if b0b1: swap r, s
```

In-place multiplication: dialog, replay, dialog backwards

```
recs, u, v = dialog(m, x, q, iters)                    # 1: x -> dialog
clear_dialog_end(m, u, v)

s = m.anc(n, "bz")                                     # 2: replay onto (y, 0)
bezout_replay(m, y, s, recs, q, dbl, cadd)
for a, b in zip(y, s):                                 # r is 0, s is the answer
    m.ctx.swap(a, b)
m.free(s)
...                                                    # 3: dialog -> x
for rec in reversed(recs):
    m.emit_inverse(dialog_round, m, u2, v2, rec)
A.encode_const(m.ctx, u2, q)                           # and starts at u = q
```

	invert, then multiply		dialog + replay	
p	Toffoli	qubits	Toffoli	qubits
7	1 095	64	612	39
31	2 889	99	1 378	57
61	4 068	115	**1 875**	**66**

The rebuild consumes the record: nothing is left behind. At $p = 61$: -54% Toffolis, -43% qubits, and the margin widens with n. One circuit gives both multiplicative steps of a point addition.

L4 rebuilt: Jacobian coordinates, no inversion at all

In $(X:Y:Z) \sim (X/Z^2,\ Y/Z^3)$, with the addend affine:

$$
\begin{aligned}
U_2 &= x_2 Z_1^2, & S_2 &= y_2 Z_1^3, \\
H &= U_2 - X_1, & R &= S_2 - Y_1, \\
Z_3 &= Z_1 H, & X_3 &= R^2 - H^3 - 2X_1 H^2, \\
& & Y_3 &= R(X_1 H^2 - X_3) - Y_1 H^3.
\end{aligned}
$$

Eleven multiplications, no division ([106]: the fewest of any quantum projective addition).

The catch: $3n$ qubits of garbage

A projective triple is not unique, so $(P + Q) - Q$ need not return the input triple: the input point cannot be erased. The 15 intermediates are cleaned; the input stays.

$p = 11$	Toffoli	qubits
affine, in place	4 769	83
Jacobian	**2 642**	98

45% cheaper; affordable once windows and zig-zag limit the live inputs

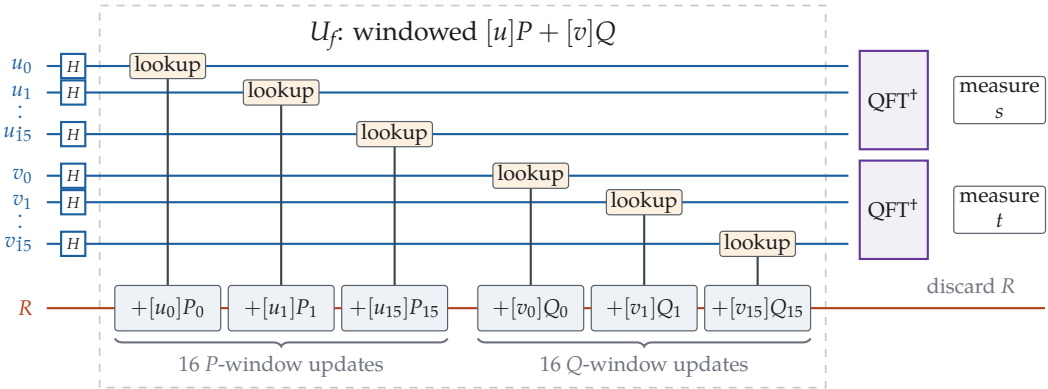
The Jacobian addition, in code: unwind out of order

```
Z1sq = A(n, "Z1sq"); _, b1 = m.step(sqr, Z1, Z1sq)                    # 1
U2 = A(n, "U2");     _, b2 = m.step(MU.modmul_const, m, Z1sq, x2, U2, p)   # 2
Z1cu = A(n, "Z1cu"); _, b3 = m.step(mul, Z1sq, Z1, Z1cu)              # 3
H = A(n, "H");       _, b4 = m.step(sub_into, U2, X1, H)              # 4
...
T0 = A(n, "T0");     _, b12 = m.step(sub_into, Rsq, Hcu, T0)          # 12
_, b13 = m.step(sub_into, T0, twoV, X3)                          # 13  OUTPUT
T1 = A(n, "T1");     _, b14 = m.step(sub_into, V, X3, T1)             # 14
T2 = A(n, "T2");     _, b15 = m.step(mul, R, T1, T2)                  # 15
T3 = A(n, "T3");     _, b16 = m.step(mul, Y1, Hcu, T3)                # 16
_, b17 = m.step(sub_into, T2, T3, Y3)                            # 17  OUTPUT
_, b18 = m.step(mul, Z1, H, Z3)                                  # 18  OUTPUT

# unwind everything but the three outputs.  Note T0 goes back via step 12,
# not step 13 -- step 13 wrote X3, which stays.
for body in (b16, b15, b14, b12, b11, b10, b9, b8, b7, b6, b5, b4, b3, b2, b1):
    m.undo(body)
```

X_3 must survive, so T_0 is released by undoing step 12, which *produced* it, not 13. The tests sweep every representative Z of each input.

L5: the circuit with its window controls expanded



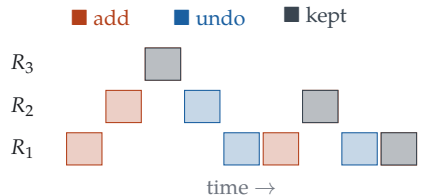
Each window register is the address of one table and leaves U_f unchanged. This is the fully coherent picture; the semiclassical version interleaves the inverse QFT with the windows and recycles the control qubits.

L5: windows, zig-zag, and one control qubit

Windowing: w control bits address a table of 2^w points: $2n+2$ additions become $2\lceil (n+1)/w \rceil$. A lookup (2^w Toffolis) is far cheaper than an addition, so $w = 16$.

Zig-zag (Grassl et al.): runs of $\rho, \rho-1, \ldots, 1$ additions on ρ registers, undoing all but the last of each run — $\rho(\rho+1)/2$ additions on ρ registers of projective garbage.

Semiclassical QFT: both scalar registers become one recycled qubit.



$\rho = 3$: six additions on three registers

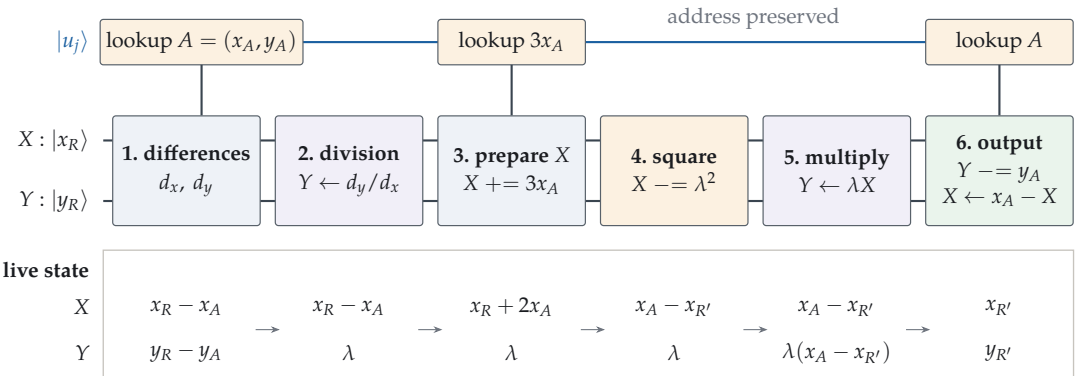
$n = 256$	none	$w = 11$	$w = 16$
point additions	514	48	**34**
zig-zag registers	32	10	8
garbage qubits	24 576		**6 144**

L5: one windowed addition is seven in-place updates

	update	after it	`windowed_padd`
1	$X \leftarrow x_R - x_A,\ Y \leftarrow y_R - y_A$	$X = d_x,\ Y = d_y$	# 3, 4
2	$Y \leftarrow Y X^{-1}$	$Y = \lambda$	# 6, division
3	$X \leftarrow X + 3x_A$	$X = x_R + 2x_A$	# 7–9
4	$X \leftarrow X - Y^2$	$X = x_A - x_{R'}$	# 10, square
5	$Y \leftarrow Y X$	$Y = \lambda(x_A - x_{R'})$	# 11, multiplication
6	$Y \leftarrow Y - y_A$	$Y = y_{R'}$	# 14
7	$X \leftarrow x_A - X$	$X = x_{R'}$	# 12, 15

Algebraic check. $\lambda = \dfrac{y_R - y_A}{x_R - x_A}$, $x_{R'} = \lambda^2 - x_R - x_A$, $y_{R'} = \lambda(x_A - x_{R'}) - y_A$. After steps 3–4, $X = (x_R - x_A) + 3x_A - \lambda^2 = x_A - x_{R'}$; steps 5–7 then give $y_{R'}$ and $x_{R'}$ with no separate output registers.

L5: the same addition as six stages on two registers



Stages 2 and 5 share one machine, a dialog GCD walk: it produces λ, then replays a fresh dialog to multiply by X. The three lookups are what Act XII makes cheap.

The windowed point addition, in code

```
MA.is_nonzero(m, addr, c[0])                # 1
_lookup(m, one[0], addr, x1, xs)            # 2
_lookup(m, one[0], addr, y1, ys)
MA.modsub(m, x1, x2, p)                     # 3
MA.modsub(m, y1, y2, p)                     # 4
_lookup(m, one[0], addr, x1, xs)            # 5
_lookup(m, one[0], addr, y1, ys)
E.inplace_div(m, x2, y2, p, iters)          # 6  y2 <- y2 / x2
_lookup(m, one[0], addr, xx, x3s)           # 7
MA.modadd(m, xx, x2, p)                     # 8
_lookup(m, one[0], addr, xx, x3s)           # 9
_csub_square(m, c[0], y2, x2, p)            # 10 if c: x2 -= y2^2
E.inplace_mul(m, x2, y2, p, iters)          # 11 y2 <- y2 * x2
MA.cmodneg(m, c[0], x2, p)                  # 12 if c: x2 <- -x2
...                                         # 13-17: add P back, unload
```

Step 6 runs the multiplier backwards, step 11 forwards: λ is consumed, not cleared by a second division.

$i = 0$ is free: O is $(0, 0)$ and steps 6 and 11 cancel. Fails only on $x_R = x_P$, $x_3 = x_P$, or $x_R = 0$ adding O.

Act VIII's optimisations in plain words (1/2)

optimisation	what it does	what it bought
L0 temporary AND	the 4-T trick inside a modular inversion	2.50× fewer T at every size
L1 Gidney adder, copy-then-add	$n-1$ ANDs instead of $2n$ Toffolis; AND the addend into a clean register under the control, add uncontrolled	controlled add 128 → 63 Toffolis ($n = 32$)
L2 top-bit, pseudo-Mersenne	decide "$\geq p$?" from a few top bits; for $p = 2^u - f$, subtracting p is clearing a bit and adding f	doubling 25 → 4 Toffolis; 40–50 top bits at size
L3 carry-save Montgomery	the reduction rewritten to look like a multiplication step: one 3:2 compressor tree does both	depth ÷2.0 ($n = 16$), 4× the qubits

Act VIII's optimisations in plain words (2/2)

optimisation	what it does	what it bought
L3 Kaliski, late reduction	run every round, reduce once at the end: doublings become wire renamings	-29% Toffolis, -41% depth ($n = 12$)
L3 Euclidean dialog, replay	keep two decision bits per GCD round, replay them linearly on any pair: inversion and multiplication in one	-54% Toffolis, -43% qubits ($p = 61$)
L4 Jacobian coordinates	projective coordinates trade the division for 11 multiplications	-45% Toffolis; $3n$ qubits of garbage
L5 windows, zig-zag, one control	16-bit windows, a schedule that bounds live garbage, and the semiclassical QFT	$514 \to 34$ additions; garbage $24\,576 \to 6\,144$

Each measured on a fixed task at a toy size, correctness re-checked in the benchmark.

Everything at once: how the savings compose

$p = 7$	Toffoli	qubits	
affine, in place	2 756	64	
Jacobian	1 544	75	−44%
windowed, dialog	1 523	49	−45%

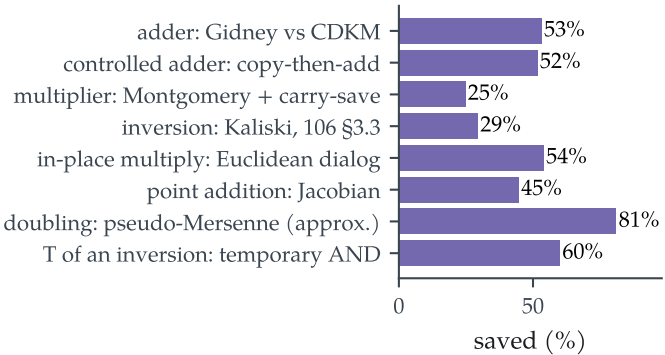
one point addition

Whole circuit, $y^2 = x^3 + 5x + 4 \bmod 7$: **69 qubits, 105 339 gates** with real arithmetic; with a permutation oracle 12 qubits, or 7 with one recycled control qubit.

- **Gate level, partly one saving**: Gidney adder and temporary AND are one idea counted twice; pseudo-Mersenne ($6.2\times$) composes.
- **Structural, pick one**: the dialog and Jacobian coordinates both answer the inversion.
- **Count level, multiply**: windows $\div 15$ additions, zig-zag $\div 4$ garbage, the semiclassical QFT deletes both counting registers.

> *Halving a point addition is worth half as much as running fifteen times fewer of them.*

ECDLP optimisations, measured



From eprints 2026/106 and 2026/1128, each on a fixed task, correctness re-checked.

Pseudo-Mersenne doubling is approximate (3% failures); Jacobian coordinates leave $3n$ garbage qubits per addition; the carry-save multiplier trades $4\times$ the qubits for depth.

> ***Resolved.*** *The same circuit solves the ECDLP; the division makes its box costlier — and all Toffolis.*

Optimisations in detail — secp256k1 at full size (Act XII)

How every optimisation is checked

A switch, not a fork

Each optimisation is a switch beside the construction it refines. Switch it off and the circuit is the old one, gate for gate.

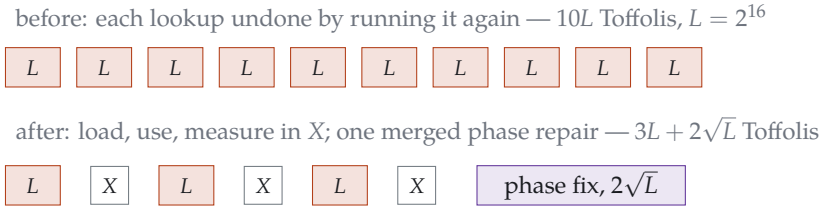
Exhaustively, at toy primes

Every basis input, every ancilla back to $|0\rangle$; a regression test pins every gate count quoted here.

Measurements leave phases

The exhaustive simulator cannot see phases, so each gate carries a unitary definition for the tests and a measurement-based *price*. A sparse state-vector simulator runs the real measure-and-repair and checks that every basis input returns with the same phase. An approximate repair fails exactly on the inputs the tests predict.

1. Lookups: ten recomputed → three loads and one repair



X-measuring a loaded register costs nothing but leaves the phase $(-1)^{m \cdot T[a]}$ on the address. A *phase lookup* of $F[a] = m \cdot T[a]$ repairs it in $\approx 2\sqrt{L}$ ANDs, against L for a lookup. IonQ's step: the three unloads share one address, so their tables are XORed and one repair serves all three.

Measured, $n = 256$**,** $w = 16$

1 310 700 → **197 364** Toffolis per addition.

2. The GCD: conditionally inverted steps

Two thirds of a point addition is the in-place multiplier, most of it the walk and replay of a binary GCD.

IonQ store $\tilde{v} = v + (1 - a)u$, $a = v \bmod 2$: both registers stay odd, the next parity is two CNOTs, and each update $\tilde{v} \leftarrow (\tilde{v} \pm u)/2$ is one *uncontrolled* adder between two rows of CNOTs.

```
def ci_add(ctx, e, x, y, anc):
    for q in y:
        ctx.cx(e, q)
    add(ctx, x, y, anc)
    for q in y:
        ctx.cx(e, q)
```

$n - 1$ ANDs, where [1128] spends $2n$ on a controlled subtraction. The recorded bits are exactly the dialog's, so its replay and packing carry over.

Measured, $n = 256$

in-place multiplication:
1 955 276 → **720 565**

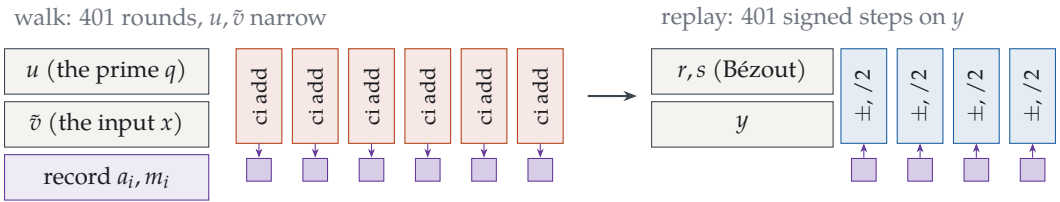
Five GCDs behind one interface

in-place multiplication, $n = 256$	rounds	Toffolis
dialog [1128], exact, full width	401	1 955 276
+ fused 77-bit compares, width schedule, pseudo-Mersenne	401	865 588
conditionally inverted (IonQ), IonQ replay	401	**720 565**
ping-pong (ECDSA.Fail)	704	917 565
Jump-2 (ECDSA.Fail)	261	1 161 099

`ec_gcd.py` holds [1128]'s dialog, IonQ's conditionally inverted walk, and ECDSA.Fail's ping-pong and Jump-2 (with its base-5 transcript codec).

> *Ping-pong is cheapest per round but needs* $2.75n$ *rounds — and exhaustively,* $n = 16$ *needs* $3.9n$.

The GCD, as a circuit: walk, record, replay



then the walk runs backwards to clear $u, \tilde{v}$, and a second walk unrecords: $x \cdot y \bmod q$ in place

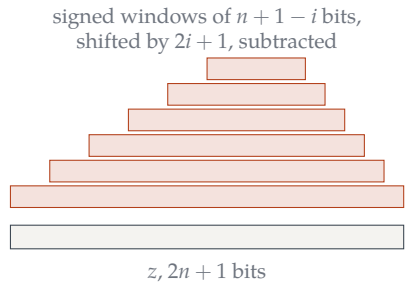
- The record is packed 5 Toffolis a round ([1128] Fig. 1) and grows into the qubits the narrowing walk frees: *register sharing*.
- IonQ's replay cells: a phase-approximate adder whose carry is the reduction flag, 352 Toffolis a step against 712.
- It cannot see $x + y = p$, met once in the first 37 steps: those keep an exact "careful" cell.

3. Squaring: a dedicated squarer and a pseudo-Mersenne fold

Step 10 subtracts λ^2; with the general multiplier that is two multiplications. IonQ's squarer:

$$x^2 = -x + \sum_i (2x_i - 1)\, 2^{2i+1} (h_i + x_i)$$

with $h_i = \lfloor x / 2^{i+1} \rfloor$. Every term is an *unconditional* signed subtraction on a window of $n + 1 - i$ bits: $\frac{1}{2} n(n+3)$ ANDs. A fold $2^{256} \equiv 2^{32} + 977$ reduces into the accumulator.



Measured, $n = 256$

square-subtract: 1 050 623 → **79 160** Toffolis.

The squarer, in code

```
for i in range(n):
    ctx.cx(z[n + i], z[n + i + 1])              # sign-extend by one bit
    h = list(x[i + 1:])
    for q in h + [pad[0]]:
        ctx.cx(x[i], q)                         # x_i ? NOT(0||h) : (0||h)
    operand = Reg(h + [pad[0], x[i]], "op")     # signed, sign = x_i
    window = Reg(z[2 * i + 1:n + i + 2], "win")
    ...
    A.sub(ctx, operand, window, anc)            # z -= signed operand
    ...
    for q in h + [pad[0]]:
        ctx.cx(x[i], q)
```

Row i: flip h by x_i so the operand is $\pm(h_i + x_i)$ with sign x_i, subtract it from the window of z at offset $2i + 1$, flip back.

Building 256 bits on a laptop: one gate per shape

One 256-bit point addition is tens of millions of gates but few distinct pieces: the replay calls one modular adder 800 times. `hier.tracing()` wraps the existing builders; each call is built once per *shape* and appended as one opaque gate.

```
def wrapper(*args, **kwargs):
    ...
    (ea, ek), qubits = sh
    key = (label, ea, ek)
    node = _CACHE.get(key)
    if node is None:
        node = _build(fn, label, key, args, kwargs, qubits, m)
        ...
        _CACHE[key] = node
    anc = m.anc(node.n_anc, "h") if node.n_anc else []
    m.qc.append(NodeGate(node), list(qubits) + list(anc))
```

Shape: register widths, shared qubits, classical parameters. Run backwards, the gate is inverted.

Checked: equal to the flat count at toy size (8 configurations); Qualtran agrees on the exported bloqs.

The first full-size builds

one windowed point addition, secp256k1, $w = 16$	Toffolis	qubits
Qualtran's `ECAdd` (Litinski 2023), for reference	8 346 972	1 798
dialog [1128], exact arithmetic, MBU lookups	5 163 617	2 104
+ 77-bit comparisons, width schedule, pseudo-Mersenne replay	2 972 327	2 251
+ dedicated squarer and fold	2 000 864	2 251
conditionally inverted walk and replay (IonQ), squarer	**1 711 194**	2 251
ping-pong walk, 704 rounds (ECDSA.Fail), squarer	2 117 534	2 155
dialog, register sharing, Fig. 1 packing, squarer	2 006 184	**1 873**

The whole algorithm builds too: 28 windowed additions, 47 978 966 Toffolis on 2 251 qubits. On the same tree Qualtran counts 249 846 912 Cliffords and 29 873 480 measurements.

> *Two gaps remain: the best row is* 23% *above IonQ's count, and every row needs more qubits than any paper reports.*

4. Matching IonQ: the gap is in the replay

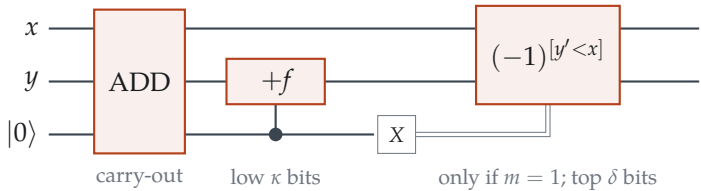
$n = 256$, Toffolis	before	after
lookups, repair	197 364	197 364
GCD walks, undone	593 432	593 432
Bézout replays	833 312	**564 086**
of which signed additions	569 600	307 574
square-subtract	79 160	79 160
everything else	7 926	4 571
one point addition	1 711 194	**1 438 613**

The walk is already IonQ's; the replay is not. Each of its 800 steps adds $\pm r$ into s modulo p, and the cell that did it costs **712** Toffolis — of which only 256 are the addition.

The rest are top-bit tests: does the sum need reducing, is it exactly p, was the input 0?

After: 1 426 965 executed, within 2.5% of IonQ's 1 392 608. The 28-addition algorithm: 40 346 698 Toffolis, against IonQ's 39.0×10^6.

IonQ's modular adder: the carry *is* the flag



If $x + y \geq 2^{256}$ the carry is set, and subtracting p is "drop the carry, add $f = 2^{32} + 977$": only the low $\kappa = 65$ bits move. No test decides the reduction.

After the correction the carry equals $[y' < x]$: reduced, $y' = x + y - p < x$; not, $y' \geq x$. So it is X-measured, and half the time a $\delta = 32$-bit comparator repairs the phase.

> *352 Toffolis worst case, 336 counting what executes (IonQ's convention), against 712.*

IonQ's modular adder, in code

```
ax, ay = m.anc(1, "ax"), m.anc(1, "ay")
sc = m.anc(n, "sc")
A.add(ctx, x + ax, y + ay, sc)                    # ay = carry-out
m.free(sc)
cp, sc2 = m.anc(kappa, "cp"), m.anc(kappa, "sc2")
A.cadd_const(ctx, ay[0], Reg(list(y[:kappa])), f, cp, sc2)   # 2^u -> f
m.free(cp, sc2)
...
def phase(qc, qs):
    ...
    qc.h(out)                                     # |->: the CNOT kicks back
    A.lt_uint(Ctx(qc, "and"), top_bits(yy, delta), top_bits(xx, delta), out,
              anc[:delta])
    ...
MB.mbu_flag(m, ay[0], list(x) + list(y), recompute, phase, nanc=delta + 1)
```

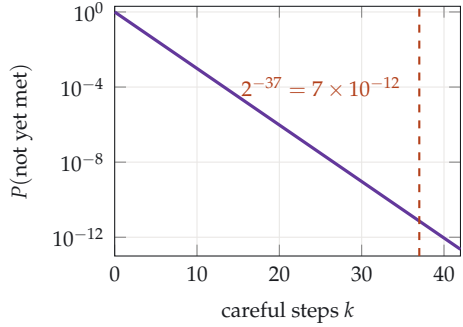
Approximate only to order f/p or $2^{-\delta}$: sums in $[p, 2^{256})$, a carry past bit κ, a tie in the top δ bits.

At toy primes: wrong on exactly the sums in $[p, 2^n)$, phase-correct everywhere else, every measurement performed.

The one case that is not rare: 37 careful steps

The replay must pass through 0. Dividing, the first swap moves a 0 into s, and complementing it gives $2^{256} - 1$, not a residue. Multiplying, the same step adds $x + y = p$ exactly: the one sum the carry cannot see.

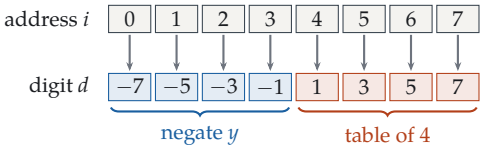
The step is fixed by the 2-adic valuation of x: within the first k steps with probability $1 - 2^{-k}$. So the first 37 steps keep the exact cell (IonQ: "rounds with $x + y = p$: 37").



measured: the failure rate halves per careful step, zero at 37

Multiplying gets its **own** replay: the inverse of "measure, sometimes repair" is "recompute", which always runs. Signed additions $569\,600 \to 307\,574$.

Signed windows: half the table, and never O



$w = 3$: $d = 2i + 1 - 2^w$, looked up by $|d|$ on $w - 1$ bits

Read the w address bits as an odd digit. The lookup runs over $|d|$; the sign is a negation of y before and after, and consecutive windows merge theirs into one, controlled by the XOR of the signs.

2^{15} entries instead of 2^{16}, and odd digits never give O.

Measured, $n = 256$

One addition: 1 340 563 Toffolis, 1 328 915 executed — 4.6% below IonQ; 2 745 400 fewer over the 28 additions.

Signed windows, in code — and the condition they need

```
for addr, table in zip(addrs, tables):
    w = len(addr)
    s, low = addr[w - 1], Reg(list(addr[:w - 1]), "l")
    ctx.x(s)                                    # s <- [d < 0]
    for q in low:
        ctx.cx(s, q)                            # low <- l~, the |d| index
    if prev is None:
        neg(m, s, y2, p)                        # R <- -R when d < 0
    else:
        ps, plow = prev                         # close the last window and
        ctx.cx(ps, s)                           # open this one: one negation,
        neg(m, s, y2, p)                        # controlled by the XOR
        ctx.cx(ps, s)
        ...
    W.windowed_point_add_cfg(m, low, x2, y2, table, p, cfg)
    prev = (s, low)
```

Needs $y \neq 0$: every visited point of odd order. secp256k1 has prime order.

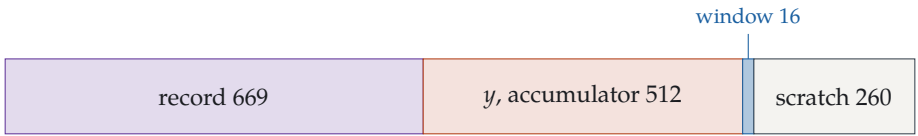
Order-43 curve, every input: one run finds k 43% of the time (ideal 77%).

IonQ's count, then IonQ's width, in one table

one windowed point addition, secp256k1	Toffolis	executed	qubits
IonQ-style walk and replay, squarer	1 711 194	1 711 194	2 251
+ phase-approximate adder in the replay	1 441 968	1 430 352	2 233
+ signed windows (2^{15}-entry tables)	1 340 563	1 328 915	2 233
+ packing, sharing, lean cells, CNOT ends	1 338 469	1 326 821	**1 457**
+ SELECT-SWAP on the $3x$ lookup	**1 322 341**	**1 310 693**	**1 457**
published: IonQ		1 392 608	1 457

> *Whole ECDLP-256:* ***37 084 197 Toffolis*** **(*36 758 053 executed*)** ***on 1 457 qubits****, 5.9% below IonQ per addition.*

Where the 1 457 qubits go



the peak: one replay cell

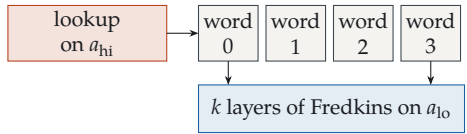
The peak moved four times, and none of the fixes adds a Toffoli:

- the careful cell's scratch is freed after its adder (259 qubits, not 647);
- the replay's opening and closing copies are n CNOTs, not exact modular additions;
- the fold's 8 subtractions run on the phase adder;
- the walk's u is shared with the record.

5. SELECT-SWAP lookups: trade qubits that are free anyway

A lookup over the high $w - k$ address bits writes 2^k words at once; k layers of Fredkins on the low bits move the addressed one into place: $2^{w-k} - 2$ ANDs $+(2^k - 1)b$ Fredkins.

The other words are a known function of the address: X-measured into the merged repair. The extra qubits live only during the load; at the $3x$ load only 798 are live, so $k = 1$ is free under the 1 457 peak.



k (points, $3x$)	qubits	Toffolis	executed
(0, 0)	1 457	1 338 469	1 326 821
(0, 1)	**1 457**	1 322 341	**1 310 693**
(1, 1)	1 565	1 290 597	1 278 949
(2, 2)	2 588	1 268 581	1 256 933
(3, 3)	4 635	1 261 413	1 249 765

SELECT-SWAP, in code

```
lam = 1 << k
lo, hi = list(addr[:k]), list(addr[k:])
...
lookup(m, hi, [q for blk in blocks for q in blk], wide)
for i in reversed(range(k)):                    # block 0 <- word a_lo
    s = 1 << i
    for j in range(s):
        for qa, qb in zip(blocks[j], blocks[j + s]):
            m.ctx.cswap(lo[i], qa, qb)
...
unlookup(m, addr, junk, G, group=group)
```

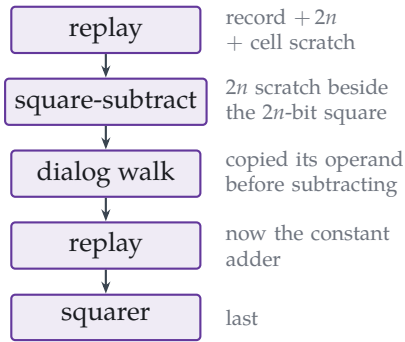
Layer i swaps block j with block $j + 2^i$ when address bit i is set: after k layers block 0 holds the addressed word. `unlookup` measures all blocks out and repairs the phases in one pass.

6. Fewer qubits: 1 246 on lean cells, 851 on three field registers

one windowed point addition; each row adds one option	qubits	Toffolis
IonQ-style (cond.-inverted walk, PM arithmetic)	2 251	1 711 194
+ Fig. 1 packing, replay in x's qubits	1 863	1 716 514
+ CDKM (1-ancilla) adders in replay, squarer, point adds	1 389	2 335 990
+ CDKM dialog walk, lean replay cells, CDKM squarer	1 246	2 935 857
+ Gidney adders where there is headroom, SELECT-SWAP	**1 246**	**2 821 091**
+ Luo's register-shared EEA instead of the dialog	1 107	86 033 905
three field registers (Luo et al. Sec. 5), signed windows	**851**	85 204 396

$1\,246 = 4.87n$: record 669, y and accumulator 512, window 16, scratch 38 ([1128]: $4.355n + O(\sqrt{n}) = 1\,192$; IonQ 1 457; Luo et al. 835).

Chasing the peak: take the scratch out of whatever sets it



where the peak sat, in the order it moved

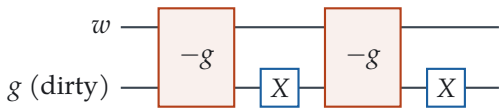
Each new cell gives the output of the one it replaces on every input tried, failures included.

The qubit count is a statement about *one moment*; every other step can borrow.

The cells that come out

- CDKM adders where no ancilla is free;
- a comparator on the CDKM carry chain: 1 ancilla, not k;
- an all-ones test in $\sqrt{k}$ chunks, ANDs parked in flags: $2\sqrt{k}$ ancillas, not $k-1$;
- a constant adder over the 33 bits of f, its carry added above by an **increment on borrowed qubits**.

An increment on borrowed qubits



$$w - g - \overline{g} = w + 1 - 2^{|w|}$$

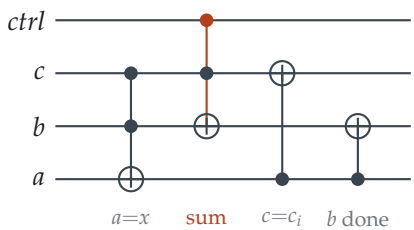
for g in *any* state

$g + \overline{g}$ is all ones, so two subtractions of a dirty register add 1 to w, and g ends as it began (Gidney 2015).

No clean scratch: the qubits are borrowed from registers that are idle at that moment, whatever they hold.

```
def inc_borrowed(m, w, g):
    ctx, k = m.ctx, len(w)
    ...
    cy = m.anc(1, "icy")
    A.sub(ctx, Reg(g), Reg(list(w)), cy)
    for q in g:
        ctx.x(q)
    A.sub(ctx, Reg(g), Reg(list(w)), cy)
    for q in g:
        ctx.x(q)
    m.free(cy)
```

A controlled CDKM adder at $3n$ Toffolis, not $4n$



the controlled UMA, after an *uncontrolled* MAJ

The package documented the textbook $3n$, but put the control on every MAJ and every UMA: $4n$.

Compute the carries **unconditionally** (one Toffoli a bit), and unwind them with a UMA that writes the sum bit only under the control (two). With the control off, b returns to y.

One of the lean cells corrected the package.

The controlled CDKM adder, in code

```
def _cuma(ctx, ctrl, c, b, a):
    ctx.ccx(c, b, a)                    # a <- x
    ctx.ccx(ctrl, c, b)                 # b ^= ctrl (c_i ^ x)
    ctx.cx(a, c)                        # c <- c_i
    ctx.cx(a, b)                        # b = y ^ ctrl (x ^ c_i)

def cdkm_cadd(ctx, ctrl, x, y, carry):
    n = len(y)
    assert len(x) == n and n >= 1
    _maj(ctx, carry, y[0], x[0])
    for i in range(n - 1):
        _maj(ctx, x[i], y[i + 1], x[i + 1])
    for i in range(n - 1, 0, -1):
        _cuma(ctx, ctrl, x[i - 1], y[i], x[i])
    _cuma(ctx, ctrl, carry, y[0], x[0])
```

On entry to `_cuma`: $b = y \oplus x$, $c = c_i \oplus x$, $a = c_{i+1}$. The carries ripple up through plain MAJs and come down through controlled UMAs.

Below 1 000 qubits: share the lanes, vent a register

Luo's extended Euclid

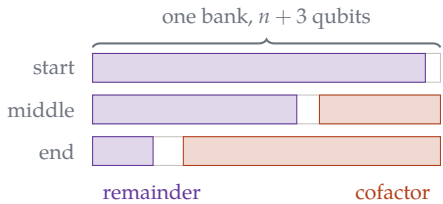
The remainders and cofactors share lanes, so the record disappears — but each quotient bit costs a shifted comparison: 30× the Toffolis for 139 qubits.

Venting a register

Once $z = y/x$ exists, y is a function of the others: it is X-measured and reused as the bank of the backward Euclid. The phase $(-1)^{b \cdot y}$ is cancelled by recomputing $y = xz$ around a Z^b.

> `ec_mbu.vent` *and* `zfix`*: the tests check that the phase cancels — and that without the* Z^b *it does not.*

The register-shared Euclidean algorithm (Luo et al.)



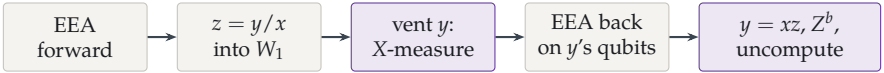
ECDSA.Fail's narrowest circuit drops the dialog: two remainders and two cofactors share two banks, the cofactors growing into the lanes the remainders free. **No record**.

The price: each quotient bit is a shifted comparison.

	inversion		division		dialog division	
n	qubits	Toffolis	qubits	Toffolis	qubits	Toffolis
32	113	374 232	177	779 682	284	32 686
64	183	1 383 179	311	2 890 248	546	120 780
256	**579**	20 433 078	**1 091**	42 836 318	2 088	**1 749 162**

Inversion: Luo's $2n + 6\lfloor \log_2 n \rfloor + 19$ exactly. Division: half the dialog's qubits, 24× its Toffolis.

Three field registers, measured



three field registers, secp256k1	qubits	Toffolis	X-meas.
in-place division (or multiplication)	**835**	42 103 470	256
controlled addition of a classical point (Fig. 14)	836	85 039 296	512
signed windowed addition, $w = 16$	**851**	85 204 396	1 792
whole ECDLP-256, 28 additions	851	2 385 774 798	
published: Luo et al. point addition	835	70.29×10^6	
published: ECDSA.Fail, narrowest	825–851		

$3n + 6\lfloor \log_2 n \rfloor + 19 = 835$ exactly, $+16$ window qubits. On Gidney's adder, exact for *every* odd p: 835 qubits, 44 200 815 Toffolis.

Three field registers, in code

```
def div3(m, x, y, p, arith, steps=None, lean=True):
    ctx, n = m.ctx, len(x)
    bk = L.Banks(m, n, x=x, lean=lean)
    L.eea_forward(m, bk, p, steps=steps)                    # W1 = 0, W2 = x^-1
    xinv, a = L._le(bk, bk.W2, 1, n), L._le(bk, bk.W1, 1, n)
    idle = bk.lt + bk.lq + bk.lr + [bk.P1, bk.P2, bk.Sign] + bk.W2[n:] + bk.W1[n:]
    with L._lent(m, idle):
        mul_acc(m, arith, xinv, y, a)                       # A = y / x
    key = ("luo3", next(_keys))
    MB.vent(m, [xinv, a], y, _quotient(p), key)             # y = A / x^-1
    bk2 = L.Banks([list(y) + bk.W1[n:]] + list(bk[1:]))      # Y is the bank now
    m.emit_inverse(L.eea_forward, m, bk2, p, None, steps, True, True)
    with L._lent(m, bk.meta() + bk.consumed + bk.W1[n:]):
        mul_acc(m, arith, x, a, y)                          # y again ...
        MB.zfix(m, y, key)                                  # ... cancels the phase
        mul_unacc(m, arith, x, a, y)
    for q1, q2 in zip(y, a):
        ctx.swap(q1, q2)
    bk.free(m)
```

Without the Z^b the same circuit fails the phase check: the check can see what the vent does.

7. Gidney's classical-quantum adder: $x \mathrel{+}= d$ on two clean qubits

The carries of $x + d$ are reproduced by adding d into the complement of the result, so a streaming adder *vents* each carry by X-measurement once its bit is summed.

The phases that leaves are undone on a *dirty* register by Z^m·carry-XOR·Z^m·carry-XOR.

A vent cannot run backwards, so subtraction and halving are built forwards; on it the three-register circuits are exact for *any* odd p.

Measured, $n = 256$

762 Toffolis $(2.98n)$ on $n - 1$ dirty + 2 clean qubits.
Luo's modular addition on it: 2 813 $(10.99n)$.

The vented adder, in code

```
for k in range(n):
    ...
        nxt = m.anc(1, "cq")[0]              # c_{k+1} = maj(x_k, d_k, c_k)
        ...
            ctx.cx(xs[k], cur)               # cur = x ^ c
            _flip(ctx, xs[k], _bit(d, k), ctrl)
            ctx.and_(xs[k], cur, nxt)        # (x ^ d)(x ^ c)
            _flip(ctx, xs[k], _bit(d, k), ctrl)
            ctx.cx(xs[k], nxt)               # maj = x ^ (x ^ d)(x ^ c)
            ctx.cx(xs[k], cur)               # cur = c again
    ...
        ctx.cx(cur, xs[k])
    _flip(ctx, xs[k], _bit(d, k), ctrl)
    if cur is not None and cur is not cin:   # vent c_k (k >= 1)
        ...
        m.free(Reg([cur]))
    cur = nxt
```

$c_{k+1} = \mathrm{maj}(x_k, d_k, c_k)$ costs one AND; c_k is vented once x_k is summed: two carry qubits suffice.

The carry-XOR, derived rather than copied

The repair needs every carry XORed into a dirty g, since $Z_q\,CX_{c\to q}\,Z_q\,CX_{c\to q} = Z_c$. With $w_k = c_k \oplus x_k$ and $y_k = x_k \oplus d_k$:

$$w_{k+1} = (x_k \oplus x_{k+1}) \oplus y_k\, w_k,$$

one product per bit. Let P be that pass on g, and Q the same pass without the constants. Then $P(g) = w \oplus Q(g)$ with Q linear and invertible, so

$$P\big(Q^{-1}(g)\big) = g \oplus w.$$

Run Q descending, then P, then XOR x_k in to turn w into c.

$\approx 2n$ Toffolis, **no ancilla**. d enters only through X and CNOT gates, so a control on the constant costs nothing.

$n = 256$	Toffolis	clean
adder, $n-1$ dirty	762	2
adder, no dirty	1 015	3
modular addition	2 813	5
modular doubling	1 535	4

$11.0n$ and $6.0n$: Luo et al. give $11n$ and $6n$

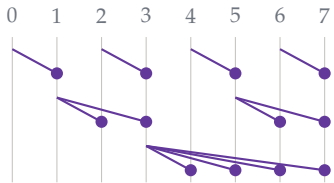
The carry-XOR, in code

```
for k in range(K):                          # y_k in place, k < K
    _flip(ctx, xs[k], _bit(d, k), ctrl)
for k in range(K - 1, 0, -1):               # Q^{-1}: g_{k+1} ^= y_k g_k
    ctx.ccx(xs[k], g[k - 1], g[k])
# P, k = 0: g_1 ^= d_0 ^ x_1 ^ y_0 (c_0 ^ d_0)
...
for k in range(1, K):                       # P: g_{k+1} ^= x_k ^ x_{k+1} ^ y_k g_k
    orig(k, g[k])
    orig(k + 1, g[k])
    ctx.ccx(xs[k], g[k - 1], g[k])
for k in range(1, K + 1):                   # w -> c: g_k ^= x_k
    orig(k, g[k - 1])
for k in range(K):
    _flip(ctx, xs[k], _bit(d, k), ctrl)
```

The adder XORs each carry into g as it goes: one carry-XOR then finishes the repair, $3n$ in all.

No dirty qubits: each half of the target serves the other, $4n$. A vent cannot be inverted.

8. Depth: carry-lookahead adders



all carries after $\log_2 n$ rounds

On an error-corrected machine, time is Toffoli depth. Ripple adders, and the GCD's sequential rounds, make depth 0.8–0.95× the Toffoli count.

[106] uses Draper–Kutin–Rains–Svore: carries as a prefix over (generate, propagate) pairs. The propagate rounds write clean ancillas: temporary ANDs, free to uncompute.

	carry-lookahead			Gidney ripple		
n	Toffolis	depth	qubits	Toffolis	depth	qubits
8	27	11	25	7	7	23
64	395	23	243	63	63	191
256	1 723	**31**	1 007	255	255	767

depth $\approx 4 \log_2 n$ for $\approx 7n$ Toffolis

Depth of one point addition: spend lookahead where the depth is

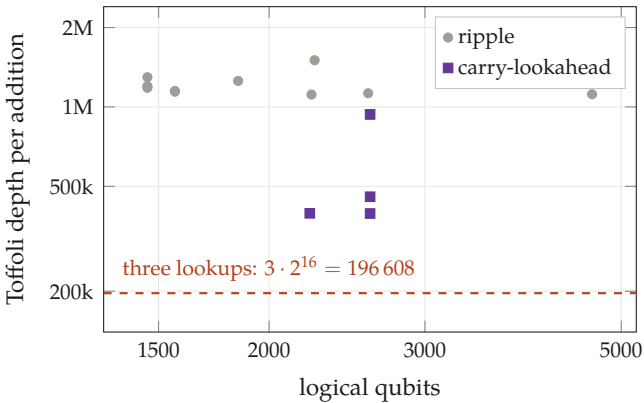
one windowed point addition, $n = 256$, $w = 16$	qubits	Toffolis	Toffoli depth
dialog, pseudo-Mersenne replay (ripple adders)	2 251	2 000 864	1 503 533
+ carry-lookahead replay	2 601	6 769 556	936 530
+ carry-lookahead walk	2 601	8 655 996	456 379
+ carry-lookahead squarer	2 601	9 184 472	394 281
+ register sharing, Fig. 1 packing	2 223	9 189 792	**394 917**

Fan out the controls. n Toffolis sharing one control take n steps: a qubit is in one gate at a time. Copy the control to n qubits with CNOTs first — no Toffoli depth.

Measured, not estimated. `hier.exact_depth` schedules the cached hierarchy gate by gate without building the flat circuit; at toy size it equals the flat depth.

> *Depth* $\div 3.8$ *for* $4.6\times$ *the Toffolis — at the same width, once the record is shared.*

The depth frontier, and the floor the lookups set



Unary iteration over 2^{16} entries is 2^{16} steps deep, so no arithmetic gets below the floor.

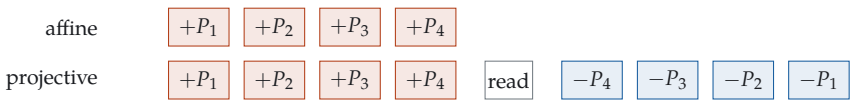
Reusing a qubit orders the steps that use it. The 1 457-qubit circuit is 33.6M deep in all; the 2 233-qubit one, 29.5M.

9. Catalogue: a looked-up addend in Jacobian coordinates

one addition, $n = 32$	qubits	Toffolis
affine, in place, controlled	540	262 797
Jacobian + classical affine	742	157 680
Jacobian + quantum affine	806	174 900
controlled	903	175 250
Jacobian + quantum Jacobian	998	257 460
controlled	1 095	257 810
to affine, once per run	571	139 066

[106] Alg. 4 with the addend looked up rather than classical (`ec_proj_q`).

A cheaper addition is not a cheaper oracle. A projective triple depends on the path that made it, so the chain must be undone before anything is read.



Undoing doubles the additions: at toy size the projective oracle costs about twice the affine one.

Catalogue: twisted Edwards curves

Extended twisted Edwards coordinates ([106] Sec. 4.4, `ec_edwards`) have a **complete** addition law: no exceptional cases at all.

7 multiplications where Jacobian needs 11.

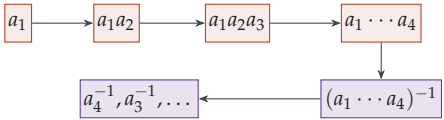
With a classical addend

3.2× fewer Toffolis and 26% fewer qubits than Weierstrass Jacobian, at both sizes.

	Edwards (Ed25519-shaped)		Weierstrass Jacobian	
n	Toffolis	qubits	Toffolis	qubits
32	**44 438**	550	143 796	742
64	**172 090**	1 094	566 132	1 478

secp256k1 is not an Edwards curve: this is a catalogue entry, not a step of the ECDLP-256 circuit.

Catalogue: batch inversion



one inversion, $3(k-1)$ multiplications

Litinski shares one inversion across k independent affine additions. Built *clean*, every intermediate uncomputed: 2 inversions $+\,5(k-1)$ multiplications, against $2k$ inversions.

			Toffolis		qubits	
n	k	what	separate	batched	separate	batched
64	2	inversions	703 732	516 346	1 054	1 118
64	8	inversions	2 814 928	**1 503 226**	1 822	2 510
256	4	inversions	22 169 576	13 414 394	5 156	5 924
256	4	point additions	65 369 012	46 815 704	5 672	7 445

Always fewer Toffolis, always more qubits — and only on the Kaliski path: in the dialog a multiplication costs as much as an inversion.

Catalogue: ECDSA.Fail's fire census

```
fired = set()
for inp in samples:
    ...
    for pos, (name, w) in enumerate(prog):
        ...
        elif name in ("ccx", "ecand"):
            if bits[w[0]] & bits[w[1]]:
                fired.add(pos)
                bits[w[2]] ^= 1
        ...
return fired
```

sample	Toffolis left	wrong
10	752	98.0%
30	891	65.2%
100	957	10.2%
300	984	2.8%
every input	991	0

the $q = 61$ dialog multiplier (1 680 Toffolis); wrong: share of fresh inputs

Run a sample, record which Toffolis ever fired, delete the rest. Over *every* input it is exact dead-code removal; over a sample it is a guess.

The dead code is the round budget's padding. At $n = 256$ that is the $2.4\sqrt{n}$ margin rounds: a census finds only what they never did *on the inputs it saw*.

10. RSA at size: Gidney 2025's residue arithmetic, loop for loop

Per prime of the residue system: look up and add the dlogs of the window multipliers, reduce mod $p - 1$, exponentiate by windowed multiplications (old results measured out), subtract the truncated CRT term into an f-bit accumulator.

n	primes	Toffolis, built	Gidney 2025	qubits, built (paper)
1024	6 242	1.18×10^9	1.1×10^9	800 (742)
2048	21 320	$\mathbf{6.95 \times 10^9}$	6.5×10^9	1 467 (1 399)
3072	47 814	1.97×10^{10}	1.9×10^{10}	2 115 (2 043)
4096	74 245	4.29×10^{10}	4×10^{10}	2 766 (2 692)

At toy N the circuit equals a bit-exact model on every exponent; at size one prime's iteration is built and multiplied by the number of primes.

ECDLP against RSA, on one surface-code model

whole algorithm (built)	logical qubits	Toffolis	physical qubits	time
ECDLP-256, IonQ cells, signed windows	1 457	3.7×10^7	2.14M	15 min
ECDLP-256, dialog, space cells	1 246	7.9×10^7	1.85M	33 min
ECDLP-256, three registers	851	2.4×10^9	1.32M	0.73 d
RSA-2048, Gidney 2025	1 467	7.0×10^9	0.97M	2.1 d
RSA-3072, Gidney 2025	2 115	2.0×10^{10}	1.26M	6.4 d

One model for every row: $p = 10^{-3}$, 1 μs cycle, 10 μs reaction, six CCZ factories. It reproduces Gidney's 897 864 qubits / 4.96 days.

> *At comparable width, secp256k1 costs* 189× *fewer Toffolis than RSA-2048 and* 537× *fewer than RSA-3072, its classical-security equal.*

Act XII in plain words (3/3)

optimisation	what it does	what it bought
Luo's register-shared Euclid	remainders and cofactors share qubit banks, so no record is kept	1 246 → 1 107 qubits for 30× the Toffolis
Three registers, venting y	measure y away once it can be recomputed; repair the phase with Z around $y = xz$	851 qubits, 85.2M Toffolis per addition
Gidney's adder, carry-XOR	add a constant on 2 clean and borrowed dirty qubits; vented carries repaired by a derived carry-XOR	762 Toffolis; exact for any odd p
Carry-lookahead, fan-out	log-depth carries; each shared control copied onto n qubits first	depth 1.50 → 0.39M, 4.6× the Toffolis

The catalogue entries (looked-up addend, Edwards curves, batch inversion, fire census) are built and measured but not in the headline circuit.

Backup — why the QFT ends with swaps

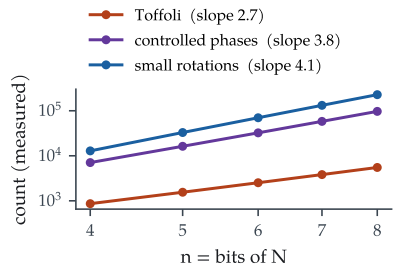
Output qubit q needs the phase $e^{2\pi i j 2^q/2^n}$, which depends only on the lowest $n-q$ bits of j. The in-place loop builds that factor on qubit $n-1-q$.

phase on each qubit ($\times 2\pi$), input $\lvert 1\rangle$, $n=3$	qubit 0	qubit 1	qubit 2
required: $j2^q/2^n$	0.125	0.250	0.500
the loop without the swaps	0.500	0.250	0.125
`qft()`, with the swaps	0.125	0.250	0.500

> *The swaps are bookkeeping (3 CNOT, 0 T): before a measurement, inside an adder, or in the semiclassical QFT the reversal is absorbed by relabelling.*

Backup — measured counts against the formulas

n	N	qubits	$4n+2$	Toffoli	$8n^3+22n^2$	CP	formula
4	15	18	18	864	864	7 068	7 068
5	21	22	22	1 550	1 550	16 245	16 245
6	33	26	26	2 520	2 520	32 322	32 322
7	65	30	30	3 822	3 822	58 107	58 107
8	129	34	34	5 504	5 504	96 888	96 888



Backup — counting in code

```
import resources as R
from shor_essentials import order_circuit

r = R.count(order_circuit(7, 15))
print(r.qubits, r.toffoli, r.cphase, r.rotations)
print(r.T, r.T_with_rotations(1e-3))
```

prints `18 864 7068 12855` and `14309 978434`.

gate	priced as
CCX, CSWAP	1 Toffoli, 7 T
MCX, k controls	$2k-3$ Toffolis
MCP, k controls	$2(k-1)$ Toffolis + CP
$\mathrm{CP}(\theta)$	2 CNOT + 3 $P(\pm\theta/2)$
$P(\theta)$, $\theta \notin \frac{\pi}{4}\mathbb{Z}$	1 small rotation
temporary AND	4 T, uncompute 0

Backup — Draper's adder in Qiskit

```
def add_const(qc, X, y):
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.p(2 * np.pi * X * 2.0 ** (i - n), y[i])
    qc.append(qft(n).inverse(), y)

def c_add_const(qc, ctrls, X, y):
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.mcp(2 * np.pi * X * 2.0 ** (i - n), ctrls, y[i])
    qc.append(qft(n).inverse(), y)
```

Backup — the modular adder in Qiskit

```
def c_add_mod(qc, ctrls, X, y, sign, flag, N):
    yw = list(y) + [sign]                    # (n+1)-wide value register
    c_add_const(qc, ctrls, X, yw)            # (1) + X          (controlled)
    add_const(qc, -N, yw)                    # (2) - N
    qc.cx(sign, flag)                        # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)           # (4) + N  if flag
    add_const(qc, -X, yw)                    # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)       # (6) clear flag from the answer
    qc.x(flag)
    add_const(qc, X, yw)                     # (7) + X
```

Backup — the ripple-carry adder in Qiskit

```python
def rc_add(qc, x, y, carry, ctrls=()):
    n = len(y)
    assert len(x) == n, "x and y must be the same width"
    _maj(qc, carry, y[0], x[0], ctrls)
    for i in range(n - 1):
        _maj(qc, x[i], y[i + 1], x[i + 1], ctrls)
    for i in range(n - 1, 0, -1):
        _uma(qc, x[i - 1], y[i], x[i], ctrls)
    _uma(qc, carry, y[0], x[0], ctrls)
```

`ctrls` reach only the two CNOTs into b of each MAJ and UMA (`rc_majuma.py`, Act VII).

Backup — the ripple-carry modular adder: control on two blocks

```
def rc_add_const(qc, X, y, anc, ctrls=()):
    n = len(y)
    carry, xreg = anc[0], anc[1:n + 1]
    Xm = X % (2**n)                       # negative X just works, as in Fourier
    for i in range(n):                    # encode the constant -- uncontrolled
        if (Xm >> i) & 1:
            qc.x(xreg[i])
    rc_add(qc, xreg, y, carry, ctrls)
    for i in range(n):                    # unencode
        if (Xm >> i) & 1:
            qc.x(xreg[i])

def rc_c_add_mod(qc, ctrls, X, y, sign, flag, N, anc):
    yw = list(y) + [sign]
    rc_add_const(qc, X, yw, anc, ctrls)  # (1) + X          (controlled)
    rc_add_const(qc, -N, yw, anc)        # (2) - N
    qc.cx(sign, flag)                    # (3) sign -> flag
    rc_add_const(qc, N, yw, anc, [flag])  # (4) + N  if flag
    rc_add_const(qc, -X, yw, anc)        # (5) - X
    qc.mcx(list(ctrls) + [sign], flag)    # (6) clear flag from the answer
    qc.x(flag)
    rc_add_const(qc, X, yw, anc)         # (7) + X
```

Backup — factoring end to end, retrying

```
while True:
    a = random.randrange(2, N)
    if (d := math.gcd(a, N)) > 1:
        return d                              # lucky gcd
    t = 2 * math.ceil(math.log2(N))
    r = order_from_counts(run(order_circuit(a, N)), a, N, t)
    if r and r % 2 == 0:
        d = math.gcd(pow(a, r // 2, N) - 1, N)
        if 1 < d < N:
            return d
```

`run`: any circuit $\rightarrow$ counts.

Backup — the garbage demo is one extra line

```
for i in range(t):
    c_ua(qc, ctr[i], pow(A, 2**i, N), list(tgt), list(anc), sf[0], sf[1], N)
if garbage == "x":
    for i in range(t):
        qc.cx(ctr[i], g[i])              # a copy of x left behind
qc.append(qft(t).inverse(), ctr)
```

Backup — ECDLP: votes for k

```
votes, q = {}, 1 << bits
for (j1, j2), c in items:
    for da in range(-search, search + 1):
        for db in range(-search, search + 1):
            a1 = (round(j1 * order / q) + da) % order
            a2 = (round(j2 * order / q) + db) % order
            if gcd(a1, order) != 1:
                continue
            k = a2 * pow(a1, -1, order) % order
            w = c / (1 + abs(da) + abs(db)) ** 2
            votes[k] = votes.get(k, 0.0) + w
return sorted(votes.items(), key=lambda kv: -kv[1])
```

Backup — ECDLP end to end

```
# the toy curve y^2 = x^3 + 5x + 4 mod 7, with G of order 5
from ec_classical import CLASSIQ as curve, CLASSIQ_G as G
from ec_shor import solve

Q = curve.mul(4, G)                          # public key: Q = [4]G
# build the circuit, run it on Aer, post-process the counts
k, counts, info = solve(curve, G, Q, order=5)
print(k, info["qubits"])
```

Backup — how the circuits are verified

Arithmetic, exactly
Every block on every basis input; every ancilla checked back to $|0\rangle$ where it is freed.

Distributions, exactly
Closed-form $P(y)$ against Aer's exact probabilities: equal to 10^{-14}.

Samples, statistically
Every histogram within the distance a perfect sampler reaches at that shot count.

> *Test the distribution, not the factors: a broken circuit gives a flat histogram, and the classical checks can still print the right factors by luck.*

Backup — elliptic curves against RSA, across key sizes

ECC bits	qubits	Toffoli	RSA bits	qubits	Toffoli
160	1 466	3.0×10^{10}	1 024	2 050	5.8×10^{11}
224	2 042	8.4×10^{10}	2 048	4 098	5.2×10^{12}
256	2 330	1.3×10^{11}	3 072	6 146	1.9×10^{13}
384	3 484	4.5×10^{11}	7 680	15 362	3.3×10^{14}
521	4 719	1.1×10^{12}	15 360	30 722	2.9×10^{15}

Rows pair comparable classical security. Minimal-width logical counts: Roetteler–Naehrig–Svore–Lauter (2017; $9n + 2\lceil \log_2 n \rceil + 10$ qubits, $(448 \log_2 n + 4090)\, n^3$ Toffolis) and Häner–Roetteler–Svore (2017; $2n + 2$ qubits). Error correction multiplies both sides by comparable factors.

Backup — reproduce every number

```
python3 -m venv venv && venv/bin/pip install -r requirements.txt
./run_tests.sh fast      # incl. semiclassical QFT and resource counts
./run_tests.sh full      # + end-to-end factoring on Aer
./run_tests.sh ec        # the ECDLP suite
venv/bin/jupyter lab notebooks/shor_walkthrough.ipynb
venv/bin/python presentation/make_figures.py    # this deck's figures
venv/bin/python presentation/make_listings.py   # its code, checked
```

Backup — references, Acts I–IX

- P. W. Shor, SIAM J. Comput. 26 (1997). S. Beauregard, Quantum Inf. Comput. 3 (2003).
- T. G. Draper, quant-ph/0008033. S. Cuccaro, T. Draper, S. Kutin, D. Moulton, quant-ph/0410184.
- R. Griffiths, C.-S. Niu, Phys. Rev. Lett. 76 (1996). C. H. Bennett, IBM J. Res. Dev. 17 (1973). T. Toffoli, ICALP 1980.
- C. Dawson, M. Nielsen, quant-ph/0505030. N. J. Ross, P. Selinger, arXiv:1403.2975.
- C. Gidney, arXiv:1709.06648 (temporary AND); arXiv:1905.07682 (windowing); arXiv:2505.15917 (2025).
- C. Gidney, M. Ekerå, Quantum 5, 433 (2021). D. Litinski, arXiv:2306.08585 (2023).
- M. Roetteler, M. Naehrig, K. Svore, K. Lauter, ASIACRYPT 2017. T. Häner, M. Roetteler, K. Svore, arXiv:1611.07995.
- Kim et al., eprint 2026/106. A. Schrottenloher, eprint 2026/1128. NIST SP 800-57; FIPS 203, 204.

Backup — references, the optimisations (Acts VII, VIII, XII)

- **Factoring.** C. Zalka, quant-ph/9806084; quant-ph/0601097 (coset representation). C. Gidney, M. Ekerå, Quantum 5, 433 (2021). R. Griffiths, C.-S. Niu, quant-ph/9511007. A. Barenco et al., quant-ph/9601018.
- S. Cuccaro, T. Draper, S. Kutin, D. Moulton, quant-ph/0410184. C. Gidney, Quantum 2, 74 (2018) (temporary AND); arXiv:1905.07682 (windows). R. Babbush et al., Phys. Rev. X 8, 041015 (2018) (unary iteration).
- **ECDLP.** H. Kim et al., eprint 2026/106. A. Schrottenloher, eprint 2026/1128. T. Häner, S. Jaques, M. Naehrig, M. Roetteler, M. Soeken, PQCrypto 2020. T. Khattar et al., arXiv:2510.10967. M. Grassl et al., PQCrypto 2016 (zig-zag).
- **Act XII.** IonQ, arXiv:2609.05625. ECDSA.Fail, arXiv:2609.09582. H. Luo et al., arXiv:2607.13816. D. Litinski, arXiv:2306.08585. G. H. Low, V. Kliuchnikov, L. Schaeffer, arXiv:1812.00954.
- T. Draper, S. Kutin, E. Rains, K. Svore, quant-ph/0406142 (carry-lookahead). C. Gidney, *Constructing large increment gates* (2015); arXiv:2507.23079. T. Häner, M. Roetteler, K. Svore, arXiv:1611.07995.

Backup — references, Act X (Hamiltonians and hardware)

- R. W. Brockett, SIAM J. Control 10 (1972). V. Jurdjevic, H. Sussmann, J. Diff. Eq. 12 (1972). D. D'Alessandro, *Quantum Control and Dynamics* (2021).
- N. Khaneja, S. Glaser, quant-ph/0010100. B. Kraus, J. I. Cirac, quant-ph/0011050. J. Zhang et al., quant-ph/0209120 (Weyl chamber). R. Wiersema et al., npj QI 10, 110 (2024).
- J. Koch et al., cond-mat/0703002 (transmon). F. Motzoi et al., 0901.0534 (DRAG). E. Magesan, J. Gambetta, 1804.04073 (cross-resonance). Google, Willow spec sheet (2024).
- A. Sørensen, K. Mølmer, quant-ph/9810039. A. C. Hughes et al., arXiv:2510.17286 (99.99%). A. Ransford et al., Nature 655, 81 (2026) (Helios).
- D. Jaksch et al., quant-ph/0004038. H. Levine et al., 1908.06101. S. Jandura, G. Pupillo, 2202.00903. S. Evered et al., Nature 622, 268 (2023); arXiv:2604.25987.
- D. Loss, D. DiVincenzo, cond-mat/9701055. P. Steinacker et al., Nature 646, 81 (2025). E. Knill, R. Laflamme, G. Milburn, quant-ph/0006088. PsiQuantum, Nature 641, 876 (2025).
- S. Puri et al., 1605.09408 (Kerr cat). H. Putterman et al., Nature 638, 927 (2025). Y. Wu et al., 2201.03540 (erasure). Tutorial §9–§10 and Appendix C: derivations and numerical checks.

Backup — references, Acts XI–XII

- **Act XI.** Babbush et al., arXiv:2603.28846. Schrottenloher, arXiv:2606.02235. Luo et al., arXiv:2607.13816. ECDSA.Fail, arXiv:2609.09582. IonQ, arXiv:2609.05625.
- Webster et al., arXiv:2602.11457 (Pinnacle). Cain et al., arXiv:2603.28627 (Oratomic). Bravyi et al., Nature 627 (2024). Gidney, Shutty, Jones, arXiv:2409.17595. Müller et al., arXiv:2506.01779.
- **Act XII.** Gidney, arXiv:2507.23079. Low, Kliuchnikov, Schaeffer, arXiv:1812.00954. `shor-complete.tex` Parts VII–VIII; `bench/`.